

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION
TECHNOLOGY



SMART BOARDING HOUSE MANAGEMENT SYSTEM

BACHELOR THESIS

Student: Nguyễn Văn Linh
Student ID: 22BI13252
External Supervisor: Lê Văn Sơn
Internal Supervisor: Trần Giang Sơn

Hanoi, June 2026

To whom it may concern,

I, (supervisor's name), certify that the
thesis/ internship report of Mr/Ms. (student's
name) is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, 28/06/2026

Supervisor's signature

ACKNOWLEDGEMENTS

I would like to express my gratitude to all those who gave me the possibility to complete this thesis, as well as the academic support that I received during the time of my studying at the University of Science and Technology of Hanoi (USTH).

Foremost, I would like to send my honest thanks to my supervisors, Assoc. Prof. Trần Giang Sơn (Internal Supervisor) and Dr. Lê Văn Sơn (External Supervisor). Their guidance, patience, and valuable feedback throughout the development of the **Smart Boarding House Management System** have been crucial to the completion of this bachelor thesis. Their insights into software architecture, database design, and systems engineering have greatly helped me improve both my academic knowledge and engineering skills.

I also want to thank the lecturers and staff of the Department of Information and Communication Technology (ICT) at USTH for providing me with a solid academic foundation and a supportive learning environment.

Finally, I would like to thank my family and friends for their constant encouragement, support, and understanding during my studies and the completion of this thesis project.

Nguyễn Văn Linh

ABSTRACT

This project is about building a website called Smart Boarding House Management System. It is a SaaS platform that helps landlords and tenants manage their rental activities together. The system solves common problems in boarding houses, such as different room information, writing contracts by hand, delay in calculating monthly bills, lack of communication between landlords and tenants, and difficulties in tracking room issues or tenant rules violations.

To build this website, NestJS is used for the backend and Next.js for the frontend. PostgreSQL with Prisma is used to store main information like rooms and contracts. MongoDB is also used to keep activity logs, and Socket.IO to send real-time notifications to users. The platform has private zones for landlords and tenants. It includes many features: managing rooms and tenants, keeping track of contracts, generating monthly invoices automatically, checking payment status, exporting PDF bills with VietQR payment codes, reporting room problems, tracking tenant violations, and using a smart AI assistant to chat with tenants or write notifications.

The result of the project is a helpful system that saves time for landlords. It also gives tenants an easy way to check their contracts, bills, payments, and to report problems. This project is a great way to learn clean coding architecture, modular frontend design, user access security, automated tasks, and how to connect AI helper to a real business system.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	ii
ABSTRACT	iii
TABLE OF CONTENTS	vi
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	1
1 INTRODUCTION	2
1.1 Context and Motivation	2
1.2 Project Overview	2
1.3 Report Structure	3
2 OBJECTIVES	4
2.1 Desired Features	4
2.1.1 Public Rental Interface	4
2.1.2 Landlord Administration Dashboard	4
2.1.3 Tenant Portal	4
2.2 Project Scope	5
2.3 Engineering Goals	5
2.4 Expected Outcomes	5
3 REQUIREMENTS ANALYSIS	6
3.1 Overall System Requirements	6
3.1.1 Public Visitor Requirements	6
3.1.2 Landlord Requirements	6
3.1.3 Tenant Requirements	6
3.2 Use Cases	7
3.2.1 Landlord Use Case	7

3.2.2	Tenant Use Case	8
3.3	Use Case Specifications	9
3.3.1	Tenant Portal Use Cases	10
3.3.2	Landlord Dashboard Use Cases	15
3.4	Data Requirements	22
4	METHODOLOGY	24
4.1	Tools and Techniques	24
4.1.1	Backend Services	24
4.1.2	Frontend Portal	25
4.1.3	Supporting Services and Integrations	25
4.1.4	Infrastructure and Deployment	26
4.2	System Architecture	26
4.2.1	Overall System Structure	26
4.2.2	Layered Design	27
4.3	Database Design	28
4.3.1	Entity-Relationship Diagram	28
4.3.2	Core Database Tables	29
4.3.3	Multi-Tenant Data Privacy	30
4.4	Authentication and Authorization	30
4.4.1	User Authentication with JWT	30
4.4.2	API Protection with Auth Guards	32
4.4.3	Multi-Tenant Authorization Filter	32
4.4.4	Room and Tenant Management	33
4.5	Automation and Business Logic	35
4.5.1	Rental Contract Creation	35
4.5.2	Invoice Payment and Verification	37
4.5.3	Tenant Incident Handling	40
4.6	AI Integration	44
4.6.1	AI Assistant with Google Gemini	44
5	RESULTS AND DISCUSSION	47
5.1	Landlord Administration Dashboard and Room Management	47
5.2	Public Room Listing and RentBot AI Chatbot	48
5.3	Tenant Portal and Personal AI Assistant	49
5.4	Invoice Payment with VietQR Integration	51
5.5	Difficulties and Limitations	52
5.6	Discussion	53

6	CONCLUSION AND FUTURE WORK	55
6.1	Conclusion	55
6.2	Future Work	55
6.2.1	Production Payment Integration	55
6.2.2	Advanced Utility Billing	56
6.2.3	Stronger Multi-Tenant Security	56
6.2.4	Mobile Tenant Experience	56
6.2.5	Analytics and Forecasting	56
6.2.6	Document and Contract Management	56

LIST OF FIGURES

3.1	Use Case Diagram - Landlord Administration Dashboard	7
3.2	Use Case Diagram - Tenant Portal	9
3.3	Sequence Diagram - Tenant Log in	11
3.4	Sequence Diagram - View Contract	12
3.5	Sequence Diagram - View Invoices	13
3.6	Sequence Diagram - Report Incident	14
3.7	Sequence Diagram - Use AI Assistant	15
3.8	Sequence Diagram - Manage Rooms	16
3.9	Sequence Diagram - Manage Tenants	17
3.10	Sequence Diagram - Manage Contracts	19
3.11	Sequence Diagram - Manage Invoices	20
3.12	Sequence Diagram - Handle Incidents	21
3.13	Sequence Diagram - Track Violations	22
4.1	Overall System Architecture	27
4.2	Database Entity-Relationship Diagram	29
4.3	Authentication and Authorization Guard Flowchart	32
4.4	Activity Diagram - Contract Creation Process	36
4.5	Activity Diagram - Invoice Payment and Verification	39
4.6	Activity Diagram - Tenant Incident Handling	42
4.7	Contextual AI Prompt Construction Flowchart	45
5.1	Landlord Administration Dashboard — Room Management Overview	48
5.2	Public Room Listing Page with RentBot AI Chatbot	49
5.3	Tenant Portal Dashboard with Contract Summary and Personal AI Assistant	50
5.4	Invoice Payment Modal with VietQR Code and Bank Transfer Details	51

LIST OF TABLES

3.1	Summary of Landlord Use Cases	8
3.2	Summary of Tenant Use Cases	9
4.1	Structure of the JSON Web Token (JWT) Payload	31
4.2	Authentication Service API Specification	33
4.3	Room Service API Specification	34
4.4	Tenant Service API Specification	35
4.5	Contract Service API Specification	37
4.6	Invoice Service API Specification	40
4.7	Incident Service API Specification	43
4.8	Violations, Rent Requests, and Supporting Services API Specification	44
4.9	Comparison of AI Assistant Modes and Data Contexts	45
4.10	AI Service API Specification	46
5.1	Summary of Implemented System Features	52

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
VND	Vietnamese Dong
WebSocket	Web-based full-duplex communication protocol

Chapter 1

INTRODUCTION

1.1 Context and Motivation

In Vietnam, many landlords of boarding houses and small rental buildings still manage their business using simple methods. They often use paper notebooks, **Excel** files, social chat applications, or check bank transfers manually. Although these methods are easy to use at first, they become very hard to manage when the landlord has more rooms and tenants. Landlords have to spend a lot of time doing things like updating room availability, collecting tenant information, writing contracts, calculating bills every month, tracking payments, answering questions, and fixing broken facilities or handling rules violations.

Using different simple tools at the same time causes many problems. For example, room information can be different between what landlords post online and what they write in their private logs. It is also difficult to find old contracts and payment history. Tenants do not have a place to check their rental information, payment status, and reported issues by themselves. Also, when landlords calculate and write bills by hand, they can easily forget deadlines, make mistakes in calculations, or send payment notifications late.

Therefore, this project aims to build a website called **Smart Boarding House Management System** to put all these tasks together in one **SaaS** platform. This system has three main parts: a **public page** to show available rooms, a **landlord dashboard**, and a **tenant portal**. The system also has automatic and smart features like creating bills on schedule, creating **PDF** invoices with **VietQR** payment codes, sending notifications, and using **AI** to chat with users or generate announcements.

1.2 Project Overview

The system has two main user groups: **landlords** and **tenants**. The first group is **landlords**, who can manage rooms, tenants, contracts, invoices, reported incidents, consultation requests, and tenant violations. The second group is **tenants**, who can view their current contract, track

their invoices, report room incidents, receive notifications, and chat with an **AI** assistant that understands their rental details.

For the **backend**, **NestJS** is used to build a modular system. The system is organized into different modules like login, rooms, tenants, contracts, invoices, rent requests, incidents, violations, PDF generation, upload, notification, and AI assistant. The **frontend** is built using **Next.js**. It is divided into public pages, login pages, a landlord admin dashboard, and a tenant portal.

1.3 Report Structure

This report is divided into six chapters:

- **Chapter 1 (Introduction):** Introduces the project, its background, reasons for development, and system overview.
- **Chapter 2 (Objectives):** Describes the objectives, functional goals, scope of work, and expected results.
- **Chapter 3 (Requirements Analysis):** Shows the requirements analysis, including user roles, system requirements, use cases, and database details.
- **Chapter 4 (Methodology):** Explains the methodology, such as the chosen technologies, application architecture, database model, and step-by-step implementation.
- **Chapter 5 (Results and Discussion):** Discusses the results, focusing on the implemented features, testing, and limitations.
- **Chapter 6 (Conclusion and Future Work):** Concludes the report and suggests some future improvements.

Chapter 2

OBJECTIVES

2.1 Desired Features

The main goal of this project is to build a website for managing rental houses. The website helps manage everything from showing vacant rooms to the public, adding tenants, writing contracts, creating monthly bills, tracking payments, reporting issues, and helping landlords and tenants communicate easily.

2.1.1 Public Rental Interface

The public page allows people who want to rent a room to look at available rooms and send requests for consultation. It only shows vacant rooms and gives details like the price, room status, and description. In addition, a smart **AI** helper can answer common questions from users by using the room data in the database.

2.1.2 Landlord Administration Dashboard

The **landlord dashboard** is the main control center of the website. It gives **landlords** tools to manage rooms, add new tenants, write rental contracts, create monthly invoices, update payment status, manage room issues, track violations, and read room requests. The dashboard also keeps data private. This means each landlord can only see their own rooms, tenants, contracts, and invoices, and cannot see information from other landlords.

2.1.3 Tenant Portal

The **tenant portal** is a personal page for renters. Tenants can log in to check their contracts, see past bills, report issues in their rooms, read violation notices, mark invoices as paid, and get support from an **AI** assistant. This makes the rental process transparent and helps landlords and tenants save time because they do not have to ask the same questions repeatedly.

2.2 Project Scope

This project is designed for small and medium boarding houses, not for large real-estate companies. The main features in this project include user login with different roles for **landlords** and **tenants**, and keeping data private for each landlord. It also allows landlords to manage rooms and upload images, manage tenant accounts, and track contract status. In addition, the system can create monthly invoices automatically, export **PDF** bills, and show **VietQR** codes based on the landlord's bank account. Landlords can also track room incidents or tenant violations. Finally, the system sends real-time notifications to users and uses an **AI** assistant to support tenants and help landlords write notices.

2.3 Engineering Goals

Apart from building the application features, this project aims to achieve several engineering goals. First, the backend uses **Clean Architecture** rules and is built with **NestJS**. Second, the frontend is built with **Next.js** and has private pages. Third, **PostgreSQL** database with **Prisma** is used to keep all data consistent. The system also uses **JWT** and security guards to protect the API routes. Moreover, scheduled cron jobs are used to create monthly invoices and check expired contracts automatically. Finally, the system connects with third-party services like **Cloudinary** for image upload, **Google Generative AI** for the assistant, and **Socket.IO** for real-time messages.

2.4 Expected Outcomes

The expected result is a working prototype of a rental **SaaS** platform. This prototype can be a good foundation to build a complete boarding house management system in the future. For **landlords**, the system will reduce manual paperwork and make it easier to see how their business is running. For **tenants**, the system will help them check their bills, payment status, and get support easily and quickly.

Chapter 3

REQUIREMENTS ANALYSIS

3.1 Overall System Requirements

The system is designed around three access contexts: **public visitors**, **landlords**, and **tenants**. Each context has different permissions and workflows.

3.1.1 Public Visitor Requirements

Public visitors do not need to log in. They must be able to browse vacant rooms, view room information, submit consultation requests, and ask basic questions through the public **AI** assistant.

3.1.2 Landlord Requirements

Landlords must authenticate before accessing the **administration dashboard**. For landlords, the system requires several core features. First, landlords need room management to create, update, delete, list, and upload images of rooms. Rooms can have status like **vacant**, **occupied**, or **maintenance**. Second, landlords need tenant management to view tenants, approve or reject tenant registration requests, and connect tenants with rooms. Third, contract management is necessary to create rental contracts, record rental price and deposit, and update contract status. Fourth, invoice management allows landlords to create bills, review payment history, check payment status, and export invoices to **PDF** files. Fifth, operational support helps landlords review room incidents, track tenant violations, and receive real-time notifications. Finally, landlords can use **AI** assistance to generate announcements and notices for tenants quickly.

3.1.3 Tenant Requirements

Tenants must authenticate before accessing private rental information. For tenants, the system requires access to private rental data after they log in. Tenants must be able to view their active and past contracts, check invoices, and see if they have paid. They also need to mark invoices

as paid so that the landlord can check and confirm their payment. If there is a problem in the room, tenants must be able to report incidents. Additionally, they can see violation notices and ask questions to the **AI** assistant about their contract or bills.

3.2 Use Cases

3.2.1 Landlord Use Case

The landlord actor manages the boarding house operation through the administration dashboard. The main workflow groups are room management, tenant management, contract and invoice processing, rent request handling, and operational support.

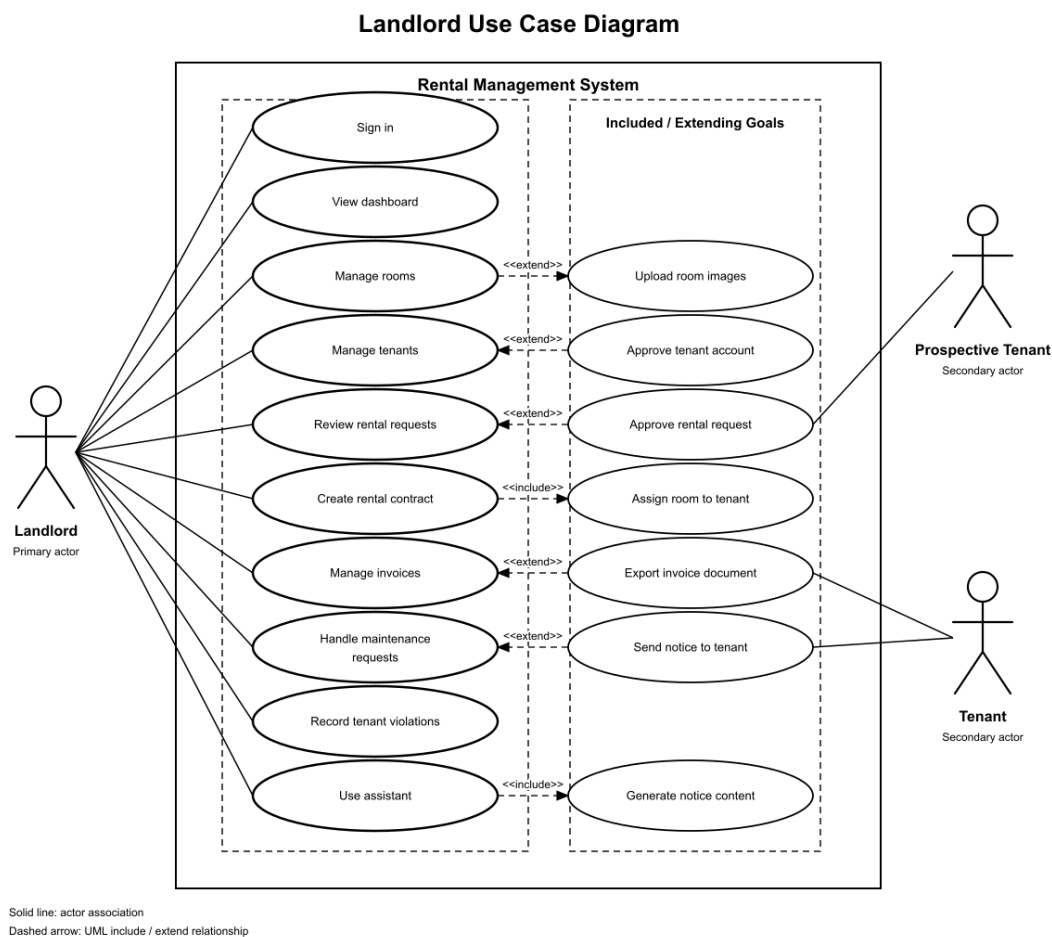


Figure 3.1: Use Case Diagram - Landlord Administration Dashboard

ID	Use Case Name	Description
UC-LAN-01	Manage rooms	Create, edit, delete, and update room availability.
UC-LAN-02	Manage tenants	Review tenant accounts and tenant information.
UC-LAN-03	Manage contracts	Create and update rental contracts.
UC-LAN-04	Manage invoices	Create invoices, update payment status, and export invoice PDFs.
UC-LAN-05	Handle incidents	Review and update tenant-reported maintenance requests.
UC-LAN-06	Track violations	Record and resolve tenant violations.
UC-LAN-07	Use AI assistance	Generate formal announcements and notices for tenants using the AI service.

Table 3.1: Summary of Landlord Use Cases

3.2.2 Tenant Use Case

The tenant actor uses the portal to monitor their rental relationship with the landlord, especially contract and payment information.

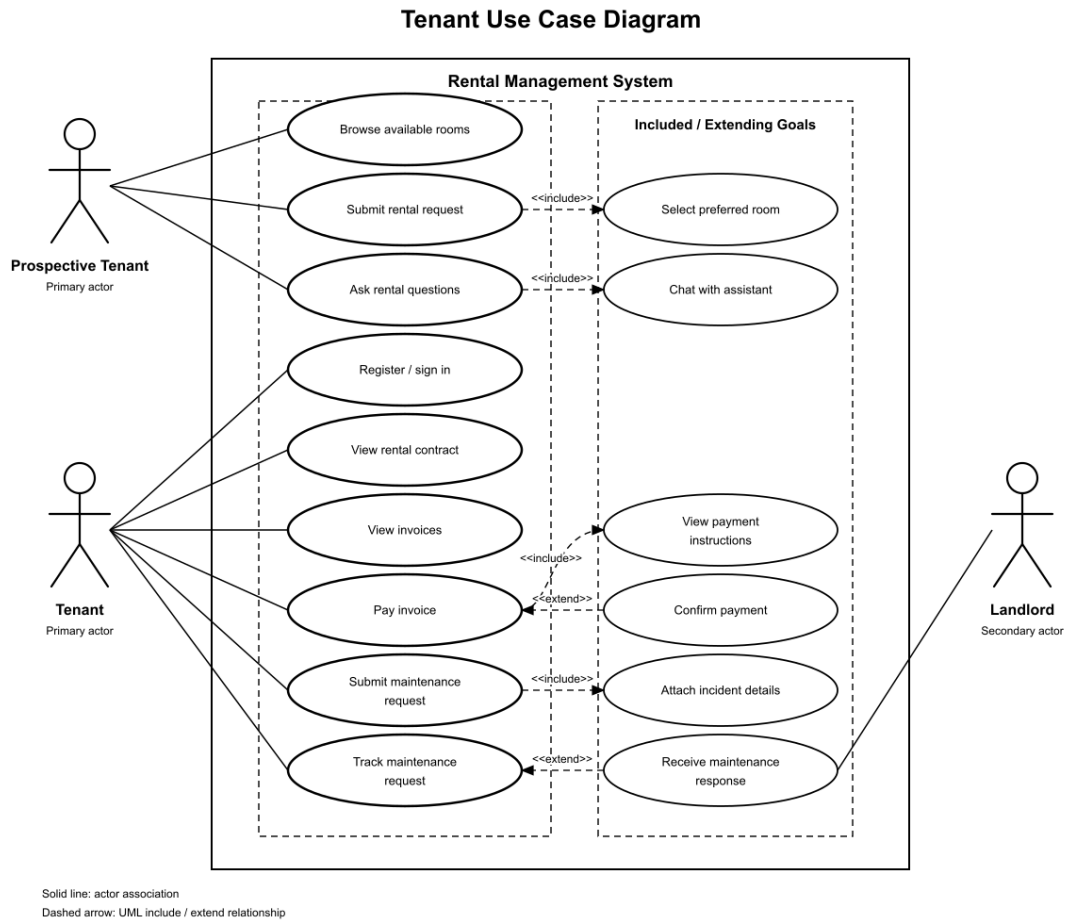


Figure 3.2: Use Case Diagram - Tenant Portal

ID	Use Case Name	Description
UC-TEN-01	Log in	Authenticate and access the tenant portal.
UC-TEN-02	View contract	Review active rental contract information.
UC-TEN-03	View invoices	Track invoices, due dates, and payment status.
UC-TEN-04	Report incident	Submit a repair or support request for the rented room.
UC-TEN-05	Use AI assistant	Ask contextual questions based on contract and invoice data.

Table 3.2: Summary of Tenant Use Cases

3.3 Use Case Specifications

This section provides a detailed description of the execution operations for specific Use Cases, outlining the interaction steps between the user and the system components.

3.3.1 Tenant Portal Use Cases

Tenant Log in

Goal: To allow tenants and landlords to log in to their account.

Primary Actor: Tenant or Landlord

Pre-conditions: The user has registered an account with a valid email and password, and has a device connected to the internet.

Main Success Scenario:

1. The user enters their email and password into the input fields.
2. The user clicks the "Log In" button.
3. The frontend validates the input format.
4. The system sends a request to the backend auth service.
5. The backend validates the credentials against the database.
6. The system generates a JWT token and returns it to the user.
7. The user is redirected to their dashboard page.

Alternative Flows: If the email or password is wrong, the system shows an incorrect password error and prompts the user to enter again. If there is a network error, the app displays a connection failure message.

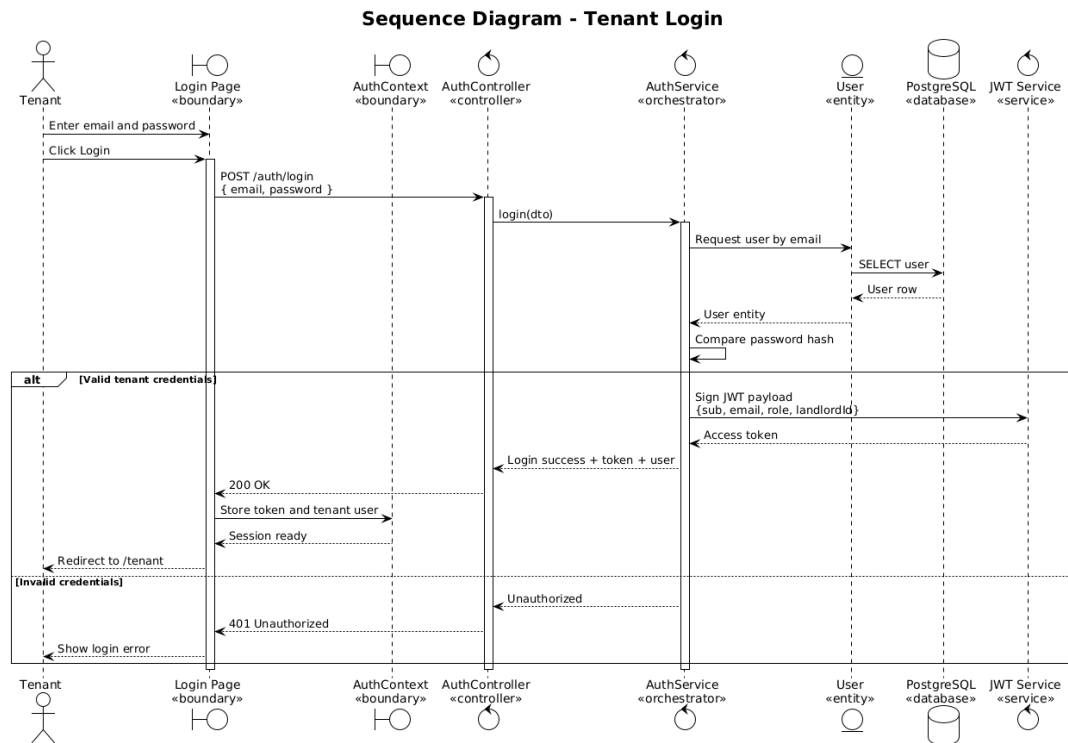


Figure 3.3: Sequence Diagram - Tenant Log in

View Contract

Goal: To allow the tenant to review their current rental contract details.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant clicks the contract tab in the portal.
2. The frontend sends a request with the JWT token to the backend.
3. The backend fetches the contract details from the database.
4. The system displays the start date, end date, rental price, and deposit.

Alternative Flows: If the contract is not found or has expired, the system shows a message that there is no active contract.

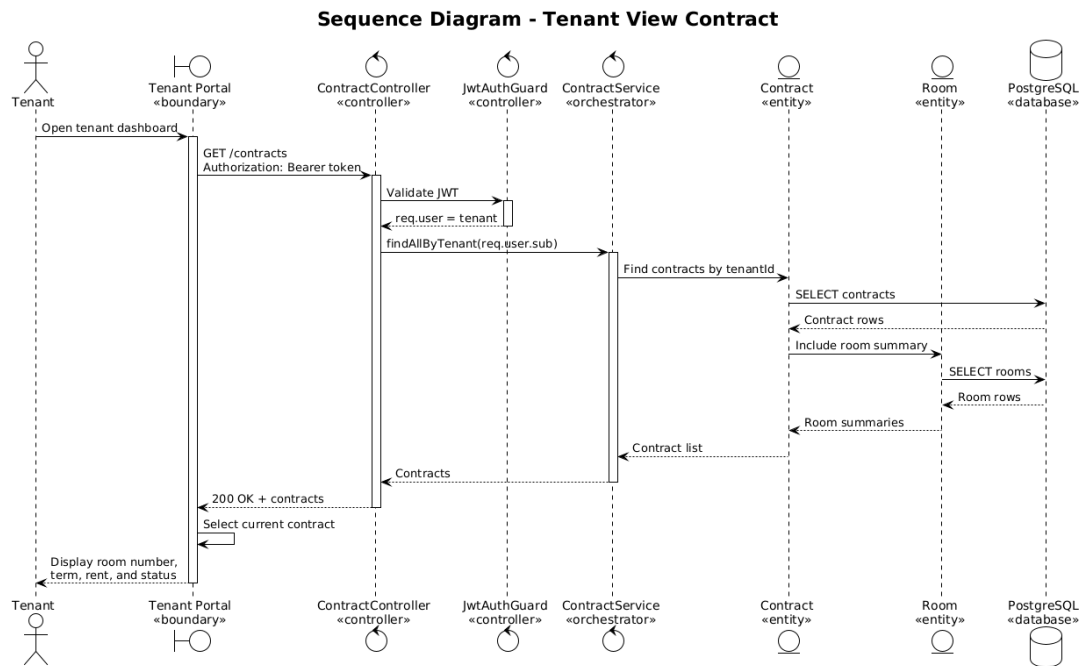


Figure 3.4: Sequence Diagram - View Contract

View Invoices

Goal: To check invoice history and payment status.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant clicks on the billing section.
2. The frontend sends a request with the JWT token to the backend.
3. The backend queries the database for all invoices related to the tenant's contract.
4. The system displays the list of invoices with their status like unpaid, paid, or overdue.

Alternative Flows: If the tenant has no invoices yet, the page shows an empty list.

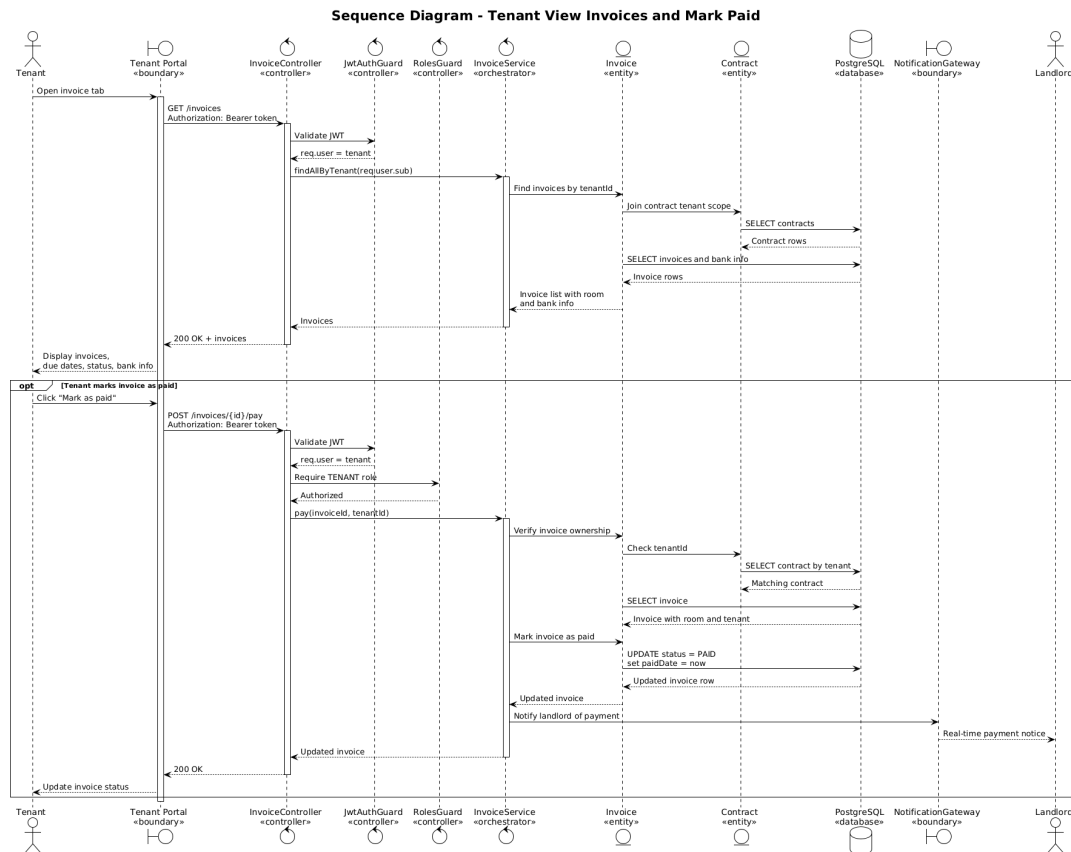


Figure 3.5: Sequence Diagram - View Invoices

Report Incident

Goal: To submit a room maintenance issue to the landlord.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant opens the incident report form.
2. The tenant enters the title, description, and selects the priority level.
3. The tenant submits the form.
4. The system saves the incident in the database and updates the status to pending.
5. The system notifies the landlord in real time.

Alternative Flows: If the description is empty, the system prevents the submission and shows a validation error.

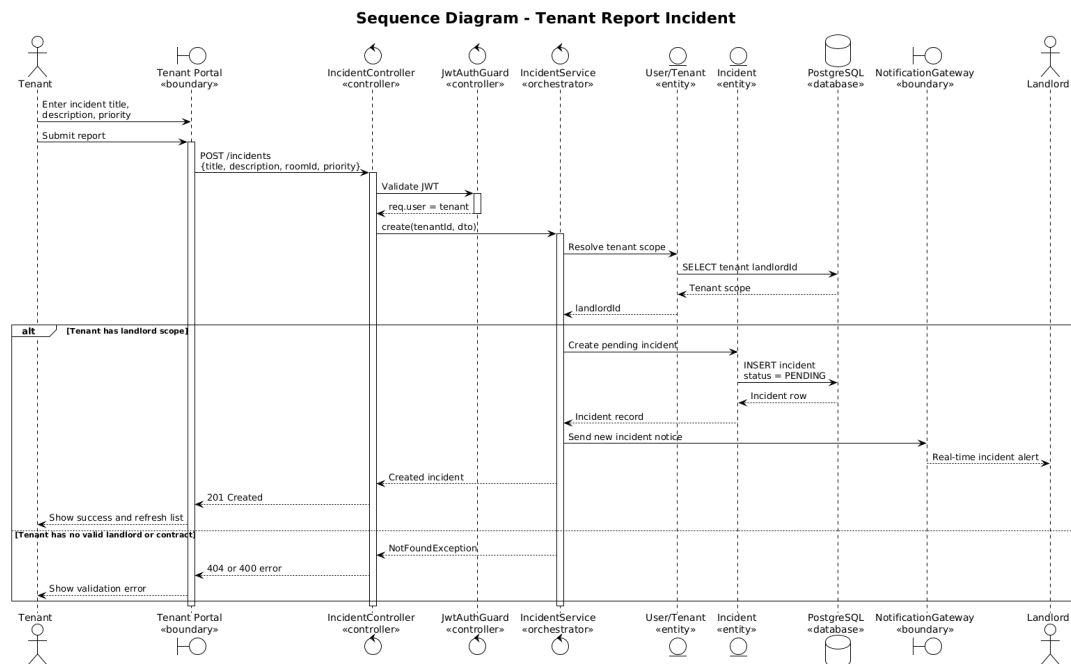


Figure 3.6: Sequence Diagram - Report Incident

Use AI Assistant

Goal: To chat with an AI assistant that knows the tenant's rental details.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant opens the AI assistant chat box.
2. The tenant writes a question about their rental terms.
3. The backend fetches the tenant's contract and invoice data.
4. The backend sends the question and the fetched data to the Google Gemini model.
5. The model generates a response, and the system displays it to the tenant.

Alternative Flows: If the AI service is busy or not working, the system returns a message asking the user to try again later.

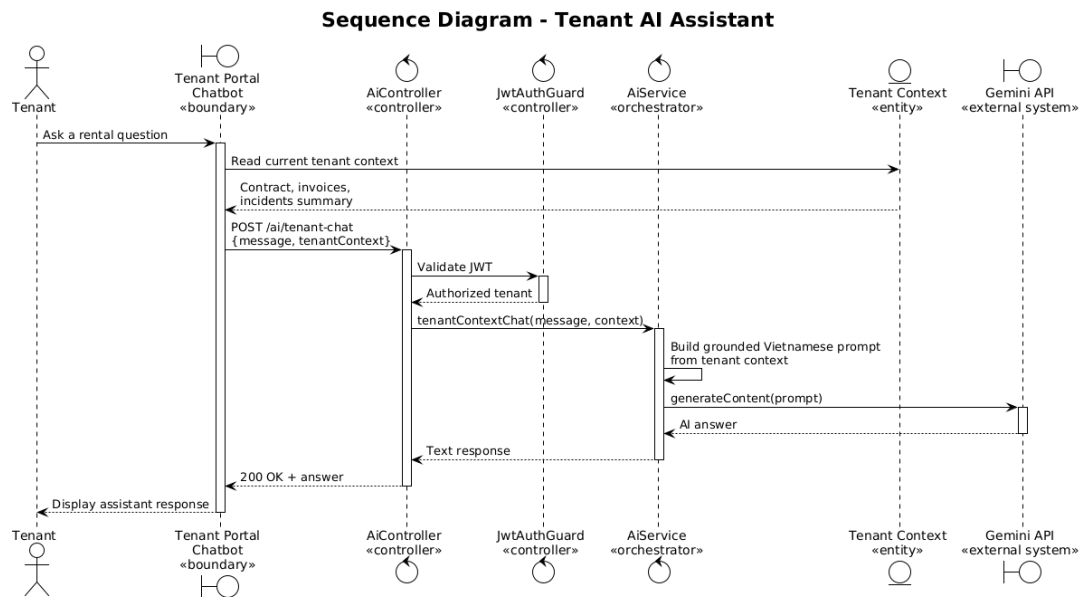


Figure 3.7: Sequence Diagram - Use AI Assistant

3.3.2 Landlord Dashboard Use Cases

Manage Rooms

Goal: To create, update, or delete room listings.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord clicks on the room management menu.
2. The landlord enters room details like room number, floor, price, description, and uploads images.
3. The landlord clicks "Save".
4. The backend saves the room information and uploads images to Cloudinary.
5. The updated room list is displayed.

Alternative Flows: If the room number already exists, the system returns a validation error.

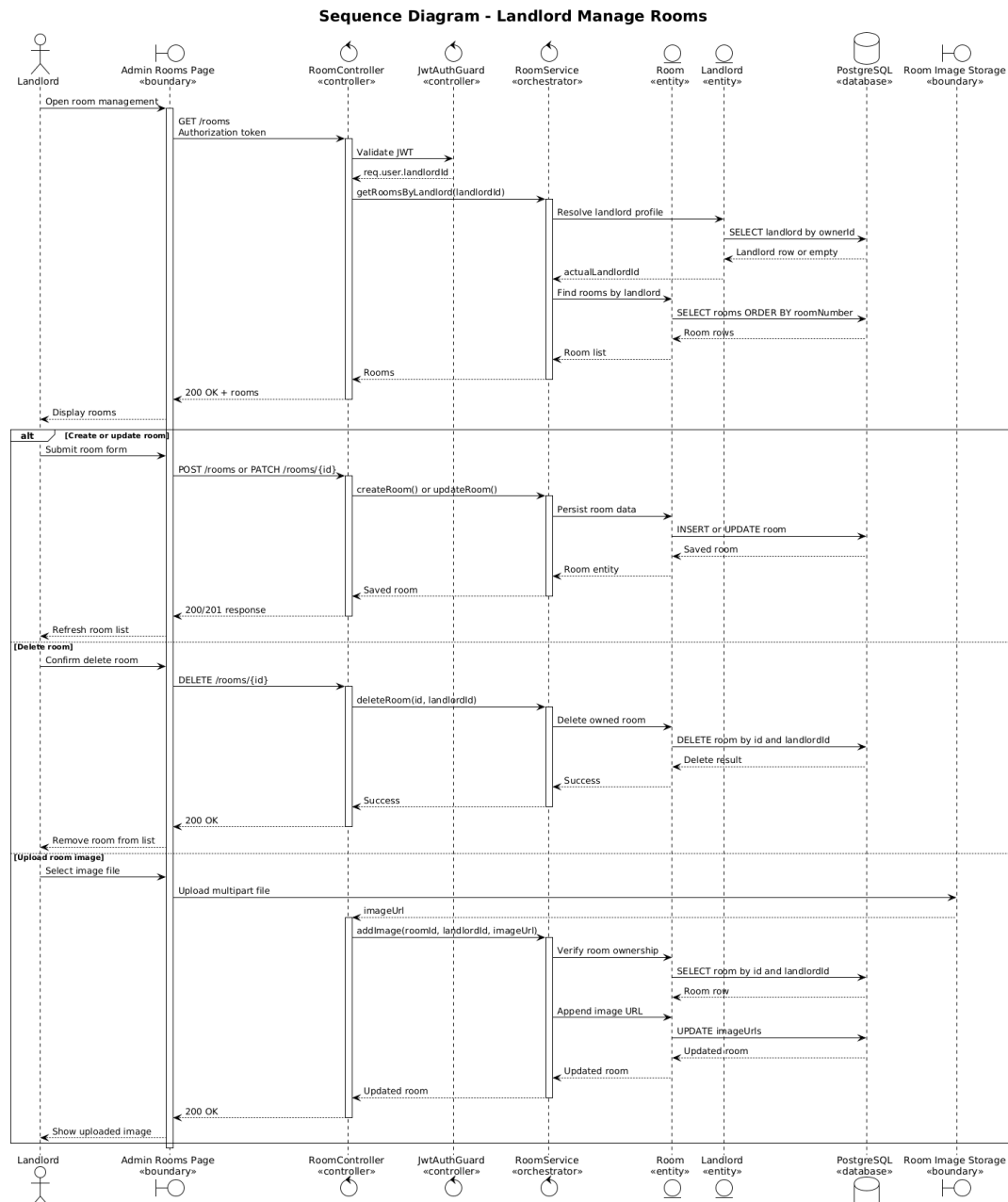


Figure 3.8: Sequence Diagram - Manage Rooms

Manage Tenants

Goal: To approve and manage tenant accounts.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the tenant management page.
2. The landlord reviews the list of registered tenants and registration requests.

3. The landlord approves a tenant.
4. The system updates the tenant status in the database.

Alternative Flows: If the tenant's information is incomplete, the landlord can reject or suspend the account.

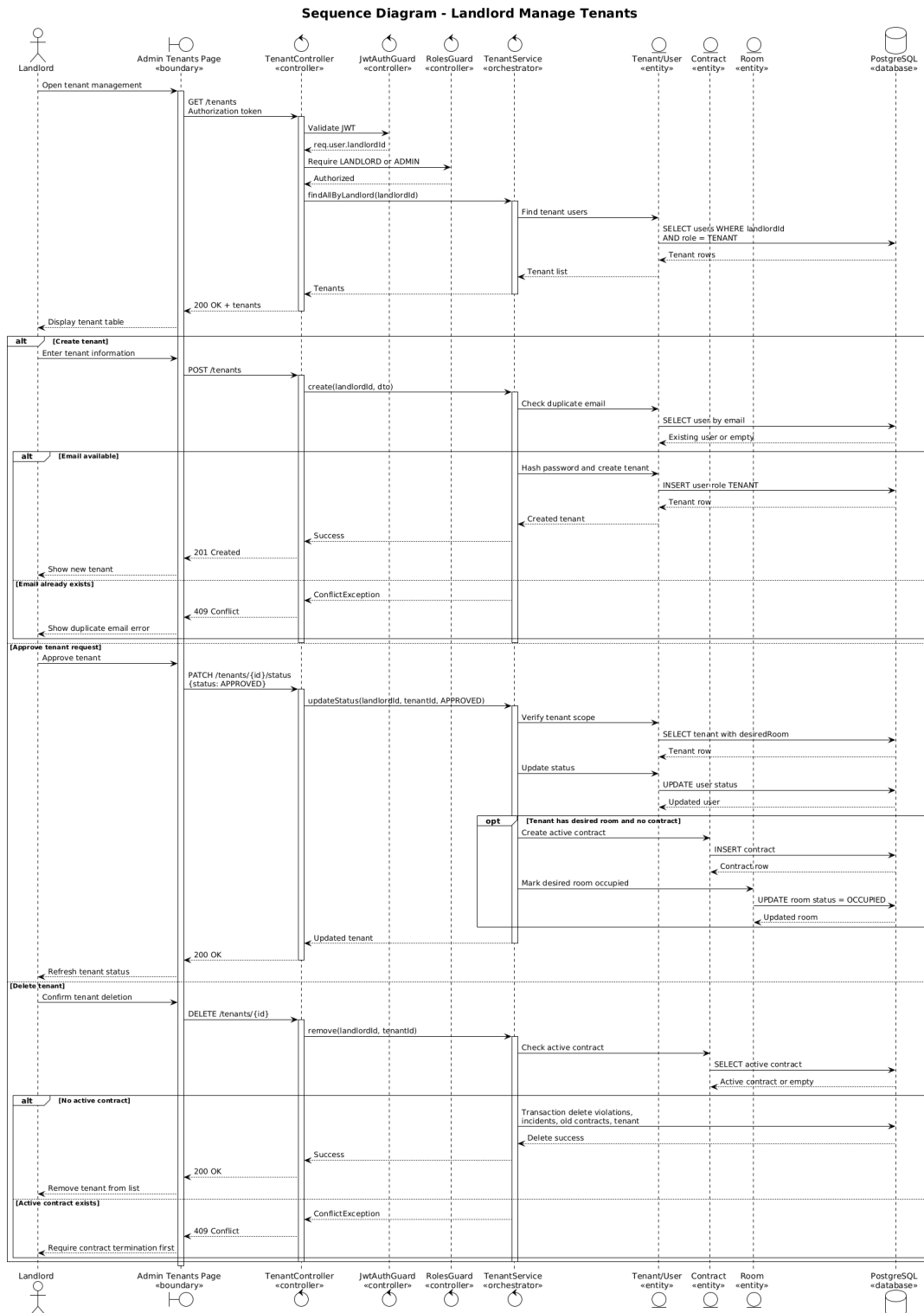


Figure 3.9: Sequence Diagram - Manage Tenants

Manage Contracts

Goal: To create a rental contract between a tenant and a room.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in. The room must be vacant, and the tenant must be approved.

Main Success Scenario:

1. The landlord enters contract details like tenant, room, dates, deposit, and rental price.
2. The landlord clicks "Save".
3. The backend validates tenant ownership and room status.
4. The system creates the contract inside a transaction.
5. The room status is updated to occupied.
6. The contract is shown in the dashboard.

Alternative Flows: If the room is occupied, the system rejects the request. If the tenant does not belong to this landlord, the system returns a validation error.

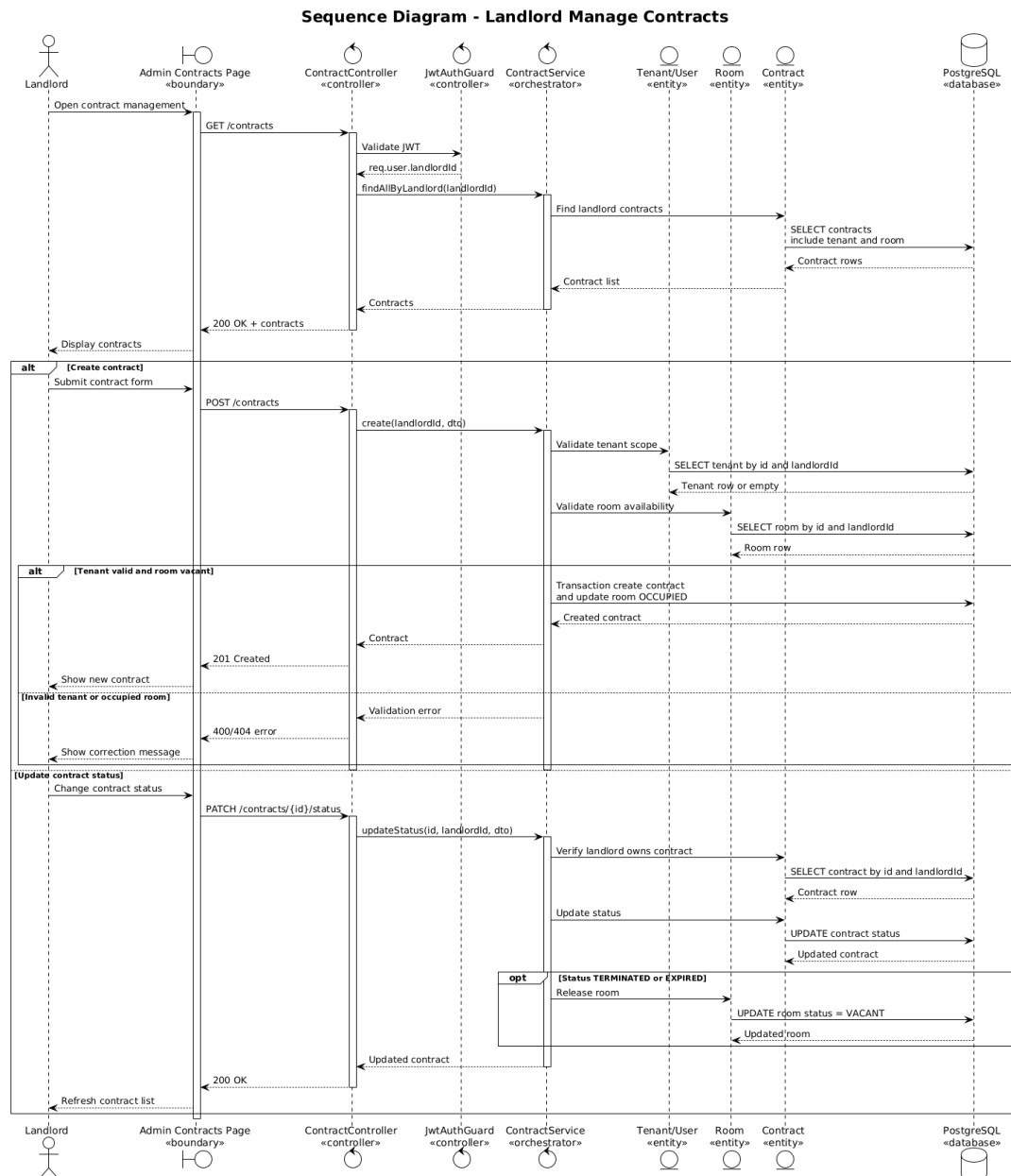


Figure 3.10: Sequence Diagram - Manage Contracts

Manage Invoices

Goal: To create invoices, check payments, and export PDF bills.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in, and an active contract exists.

Main Success Scenario:

1. The landlord goes to the billing section.
2. The landlord enters details like monthly price, electric usage, and water usage.
3. The landlord saves the invoice.

4. The system calculates the total amount, saves the invoice, and creates a PDF file.
5. The landlord can view the PDF invoice with a VietQR code.

Alternative Flows: If the invoice details contain invalid values, the system returns an error.

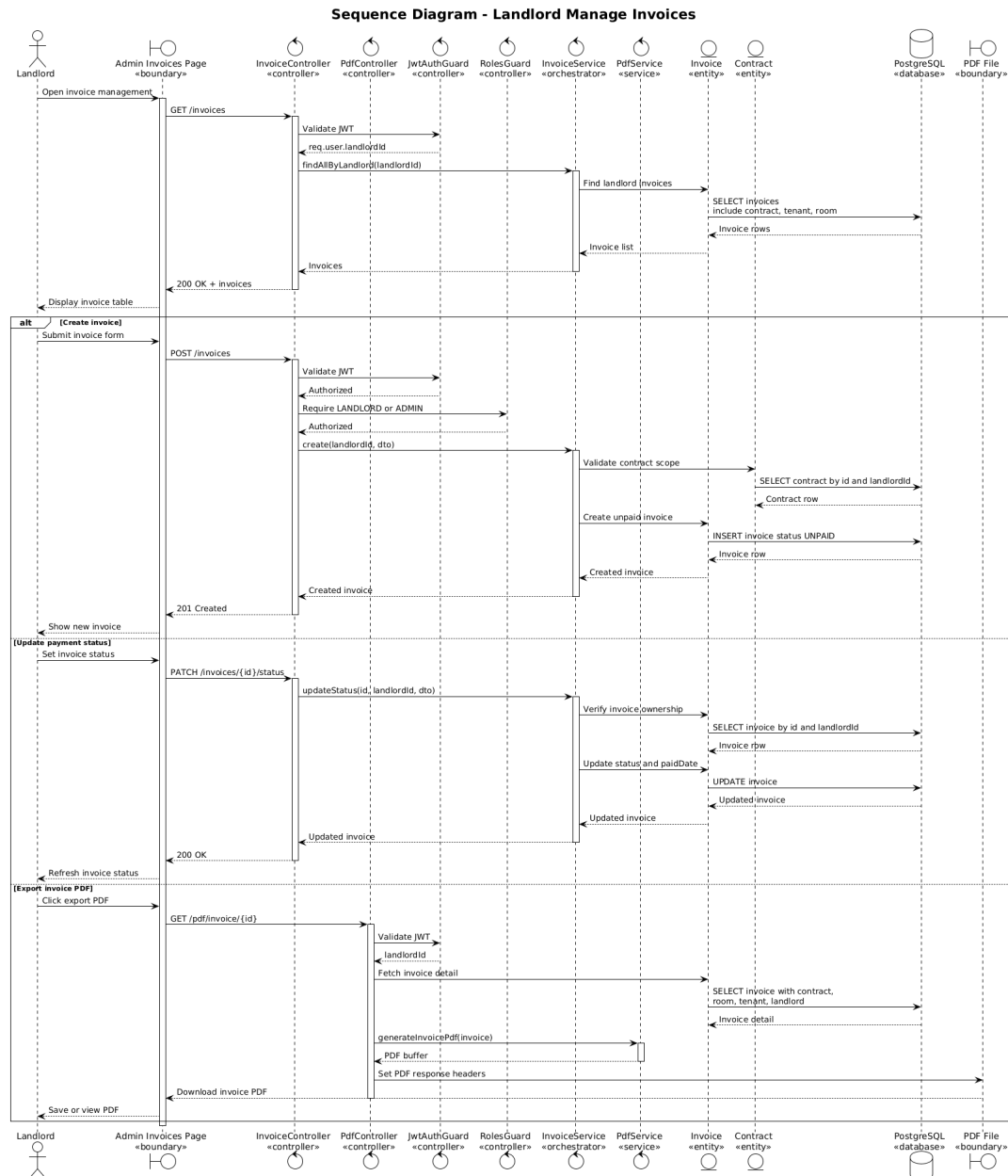


Figure 3.11: Sequence Diagram - Manage Invoices

Handle Incidents

Goal: To review and solve tenant-reported incidents.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the incident management list.
2. The landlord reviews the pending issues reported by tenants.
3. The landlord updates the incident status to "In Progress" or "Resolved".
4. The system saves the status in the database and notifies the tenant in real time.

Alternative Flows: If the incident has already been resolved by another admin, the system prevents duplicate actions.

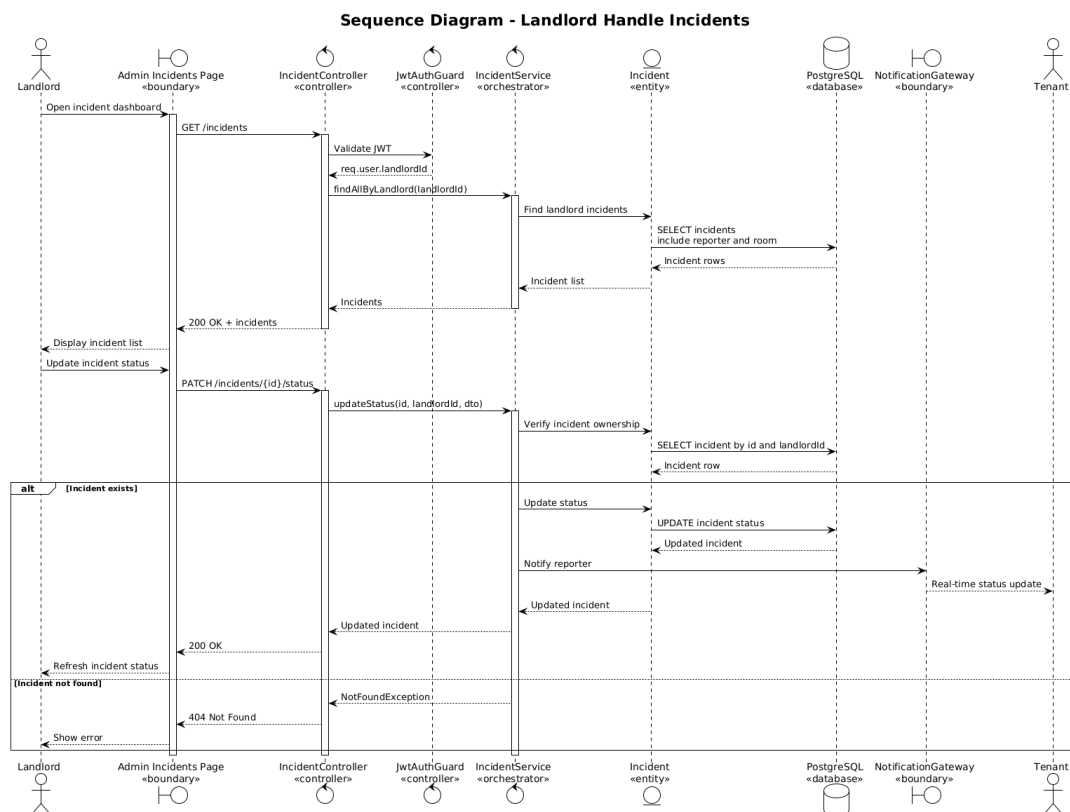


Figure 3.12: Sequence Diagram - Handle Incidents

Track Violations

Goal: To record rules violations for tenants.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the violations page.
2. The landlord enters a description of a rule violation for a tenant.
3. The landlord clicks "Add".

4. The system saves the violation record and makes it visible to the tenant.

Alternative Flows: If the target tenant does not exist or has left, the system returns an error.

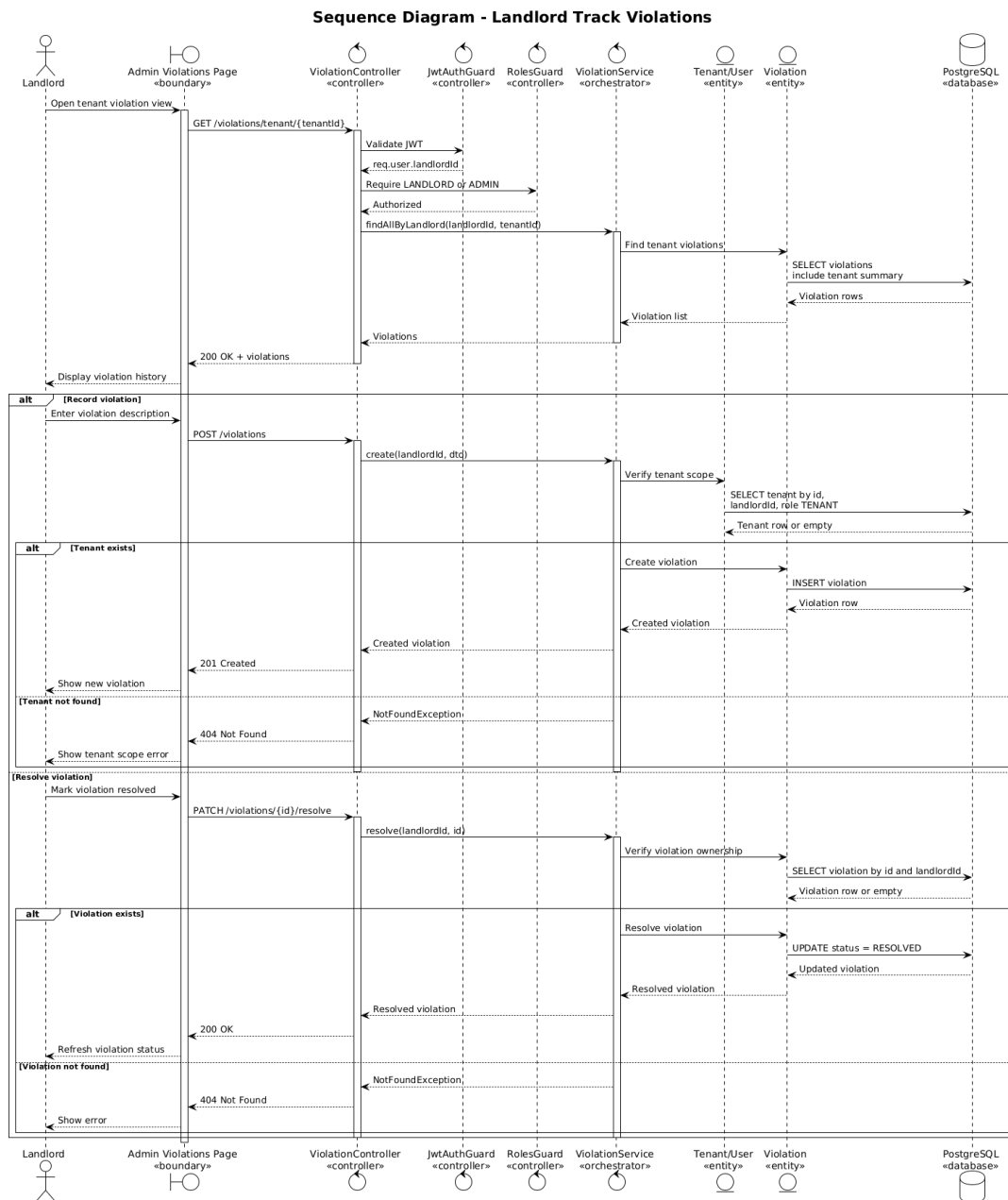


Figure 3.13: Sequence Diagram - Track Violations

3.4 Data Requirements

The core database model uses a relational structure. The main entities are users, landlords, rooms, contracts, invoices, rent requests, incidents, and violations. Each record is scoped by a landlord identification code. This keeps the data private for each landlord. Main status fields include room availability, contract status, invoice payment status, user registration status, and

incident resolution status.

Chapter 4

METHODOLOGY

This chapter presents the architecture and implementation methods used to build the *Smart Boarding House Management System*. It explains the technology choices, system design, database structure, security mechanisms, business logic automation, and AI integration in detail.

4.1 Tools and Techniques

Choosing the right tools is very important to build a reliable rental management platform. The selection of the technology stack was driven not only by functional requirements but also by specific engineering goals: ensuring data consistency, enabling real-time communication, and supporting rapid development across a modular codebase. This section explains the technologies selected and the reasons for choosing them over common alternatives.

4.1.1 Backend Services

- **NestJS (vs. Express):** While Express is a popular minimalist framework, **NestJS** is chosen because it provides a structured, modular architecture built on top of Express with built-in support for dependency injection, decorators, and guards. This structure is essential for organizing a large codebase into independent domain modules such as authentication, rooms, contracts, invoices, and AI. Without this structure, a plain Express application would quickly become difficult to maintain as the number of features grows.
- **PostgreSQL with Prisma ORM (vs. MongoDB):** **PostgreSQL** is chosen as the primary database because rental management data, particularly contracts and invoices, requires strict relational integrity and ACID-compliant transactions. For example, when a contract is created, the room status must be updated atomically in the same operation. A document database like **MongoDB** cannot enforce foreign key relationships or multi-table transactions as reliably. **Prisma ORM** is used on top of **PostgreSQL** because it provides type-safe query builders and auto-generated TypeScript types from the schema, which prevents type mismatch bugs at compile time and makes database migrations traceable.

- **Socket.IO (vs. HTTP Polling):** Instead of having the frontend repeatedly send HTTP requests to check for new notifications, **Socket.IO** is used to maintain a persistent Web-Socket connection between the server and each logged-in client. This allows the backend to push events such as invoice payment confirmations and new incident reports to the landlord's browser instantly, without any delay or wasted network traffic from polling.

4.1.2 Frontend Portal

- **Next.js 14 App Router (vs. React SPA):** **Next.js** is chosen instead of a standard single-page application because it supports server-side rendering. This is important for the public landing page, where room listings must be visible to search engines to attract potential tenants. The App Router also provides a clean file-based routing system that naturally separates the three access contexts: public pages, the landlord administration dashboard, and the tenant portal.
- **Zustand (vs. Redux):** For global state management, **Zustand** is chosen over **Redux** because it requires significantly less boilerplate code. Redux requires defining actions, reducers, and selectors separately, which adds unnecessary complexity for managing simple states such as the logged-in user session and UI preferences. Zustand achieves the same result with a simple hook-based store, making the codebase easier to read and maintain.
- **Axios with Interceptors:** For API requests, **Axios** is used with custom request interceptors. These interceptors automatically attach the **JWT** Bearer token and the landlord identification header to every outgoing request, so individual components do not need to manage authentication headers manually. A response interceptor also handles token expiry errors globally.

4.1.3 Supporting Services and Integrations

- **Cloudinary (vs. Local File Storage):** Room images are stored on **Cloudinary** instead of on the local server disk. This avoids filling up the server's disk space and allows **Cloudinary** to automatically optimize and resize images for faster loading on public devices. The backend stores only the secure image URL in **PostgreSQL**, keeping the database lightweight.
- **Google Gemini (vs. OpenAI GPT):** The **Gemini 2.0 Flash Lite** model is chosen for the AI assistant features because it offers a very fast response time and a generous free-usage tier, which keeps operational costs low for a student project. It is accurate enough for the three required tasks: generating formal Vietnamese announcements, answering room availability questions, and answering tenant billing questions from structured database context.

- **VietQR API:** The **VietQR** image API is called dynamically when an invoice PDF is generated. It produces a QR code image pre-filled with the landlord's bank account number and the exact invoice amount. Tenants can scan this code with any Vietnamese banking application to complete the transfer instantly, without needing to type account numbers manually.

4.1.4 Infrastructure and Deployment

- **Neon (Serverless PostgreSQL):** The **PostgreSQL** database is hosted on **Neon**, a serverless cloud database provider. Neon automatically scales compute resources based on demand and provides connection pooling, which is important because serverless backends on **Render** can spin up many instances simultaneously.
- **Render (Backend Hosting):** The **NestJS** backend is deployed as a Web Service on **Render**. **Render** detects the backend directory from the **GitHub** repository, runs the build command automatically, and connects to the **Neon** database through environment variables.
- **Vercel (Frontend Hosting):** The **Next.js** frontend is deployed on **Vercel** because **Vercel** is built and optimized specifically for **Next.js** applications. It handles server-side rendering infrastructure automatically and provides a global CDN for fast page loads. Both **Render** and **Vercel** are connected to the **GitHub** repository, enabling automatic **CI/CD**: any code pushed to the main branch is built and deployed to production without manual intervention.

4.2 System Architecture

This section describes how the different parts of the Smart Boarding House Management System are organized and how they connect with each other. The system is designed using a clean, layered structure to separate the user interface, business logic, and data persistence concerns.

4.2.1 Overall System Structure

The system follows a **client-server** architecture. The frontend built with **Next.js** acts as the client, and the backend built with **NestJS** acts as the server. As shown in Figure 4.1, the client and server communicate using **HTTP REST APIs** for standard data requests and **Socket.IO** WebSockets for real-time event notifications. The backend server connects directly to the **PostgreSQL** database on **Neon** to read and store all rental information. It also communicates with three external services: **Cloudinary** for image storage, **Google Gemini** for AI features, and the **VietQR API** for payment QR code generation.

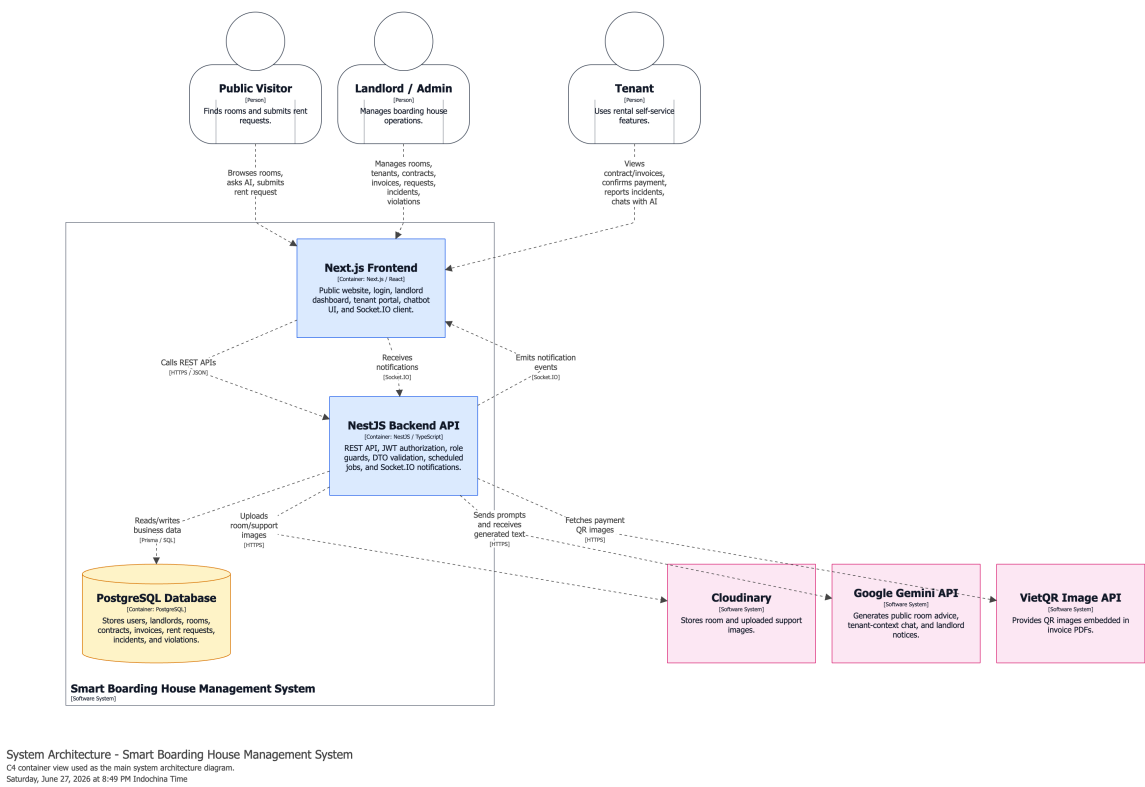


Figure 4.1: Overall System Architecture

Design Rationale: A single backend service rather than a microservices architecture is chosen deliberately. Because the project is built and maintained by a single developer within a limited timeframe, a monolithic **NestJS** application organized into domain modules provides the right balance between structure and simplicity. Each domain module (rooms, contracts, invoices, AI, etc.) is self-contained and could be extracted into a separate service in a future version without significant refactoring.

4.2.2 Layered Design

To make the codebase easy to maintain and extend, both the **backend** and **frontend** are organized into three distinct layers: the **presentation layer**, the **application layer**, and the **infrastructure layer**. This separation ensures that changes in one layer do not directly affect the others.

The **presentation layer** is the top layer. In the **backend**, it contains **NestJS** HTTP controllers and Data Transfer Objects (DTOs) that receive incoming requests, validate their shape, and send formatted responses. In the **frontend**, it contains the **Next.js** pages and **React** components that render the UI. This layer does not contain any business rules; its sole responsibility is to pass validated data to the next layer.

The **application layer** contains the core business logic. For example, when a landlord creates a contract, it is this layer that checks whether the selected room is vacant, executes the database transaction, and updates the room status. It contains service classes for authentication, room management, contract lifecycles, invoice creation, and notification dispatch. This layer is completely independent of the specific database library or HTTP framework being used.

The **infrastructure layer** is the bottom layer. It handles all communication with external systems. In the **backend**, this includes **Prisma** repository classes for executing **PostgreSQL** queries, a **Cloudinary** upload service, and the **Google Generative AI** client. This layered design ensures that, for example, switching from **Prisma** to another ORM would only require changes in the infrastructure layer without touching the business logic above it.

4.3 Database Design

The database is the foundation of the platform, storing all rental information in a structured and consistent way. **PostgreSQL** is used as the database and **Prisma ORM** manages the connection and schema. This section explains the entity relationships, the purpose of each table, and how the schema enforces multi-tenant data privacy.

4.3.1 Entity-Relationship Diagram

To understand how the data is structured, an **Entity-Relationship Diagram (ERD)** is created. As shown in Figure 4.2, the database contains eight main tables that represent the core domain objects of the boarding house system: **User**, **Landlord**, **Room**, **Contract**, **Invoice**, **RentRequest**, **Incident**, and **Violation**.

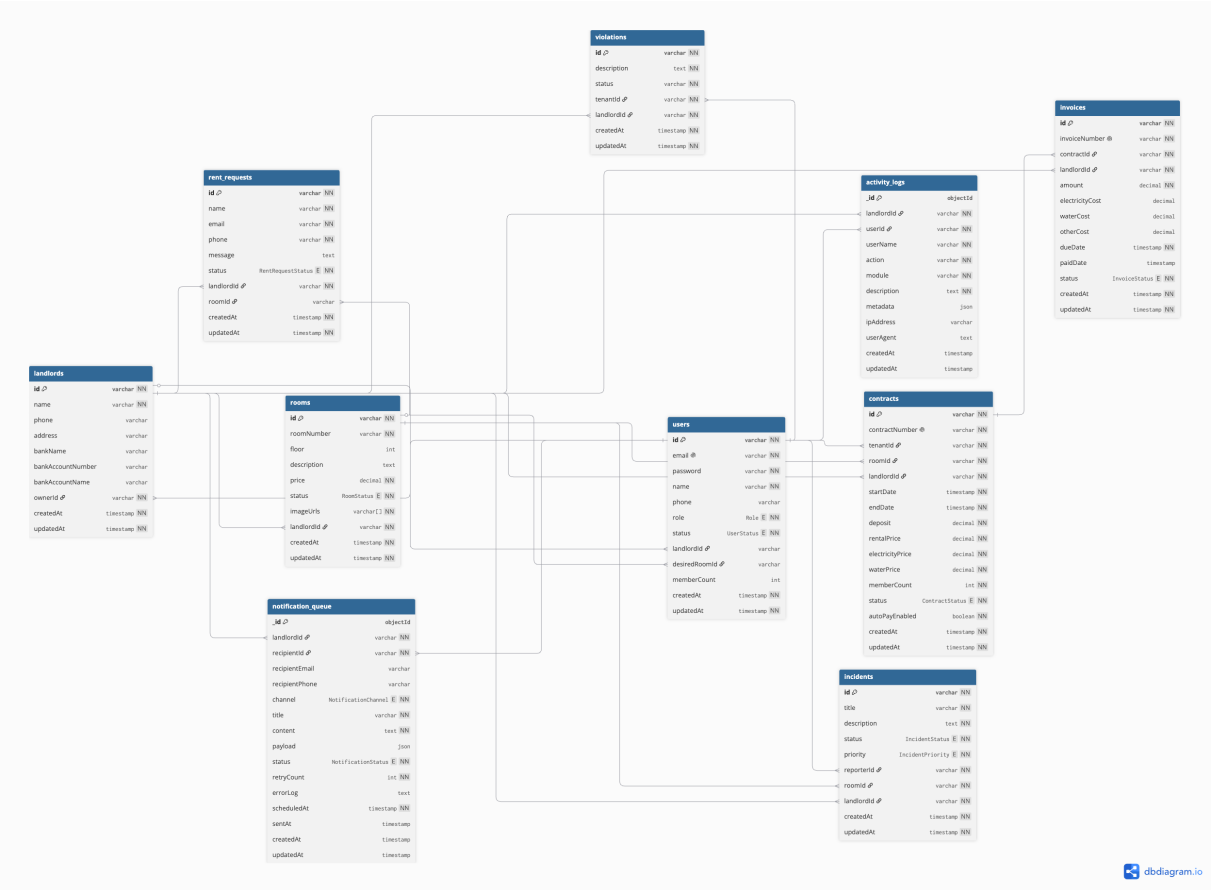


Figure 4.2: Database Entity-Relationship Diagram

4.3.2 Core Database Tables

The **User** table is the central identity record for all accounts in the system. Each user record stores an email, a bcrypt-hashed password, a display name, a phone number, an account status (**PENDING**, **APPROVED**, **REJECTED**, or **BANNED**), and a role (**ADMIN**, **LANDLORD**, or **TENANT**). If the account belongs to a tenant, the record also stores a landlordId foreign key that connects the tenant to their specific property owner. New user registrations are saved with **PENDING** status by default, so landlords must explicitly approve tenants before they can log in.

The **Landlord** table stores the business profile of each property owner. It records the landlord’s name, phone, address, and bank account details (bank name, account holder name, and account number). These bank details are critical because they are fetched automatically every time an invoice PDF is generated to produce the **VietQR** payment code.

The **Room** table represents a rentable unit. Each room record stores a room number, floor, description, a decimal rental price, a status field (**VACANT**, **OCCUPIED**, or **MAINTENANCE**), and an array of Cloudinary image URLs. Every room is associated with a landlord through a landlordId foreign key with cascade deletion, meaning all room records are automatically

removed if the landlord account is deleted.

The **Contract** table represents the formal lease agreement between a landlord, a tenant, and a room. It stores the contract number, start date, end date, deposit amount, monthly rental price, electricity unit price, water unit price, member count, and a status field (**PENDING**, **ACTIVE**, **EXPIRED**, or **TERMINATED**). The contract also stores a boolean `autoPayEnabled` flag that controls whether the monthly cron job should automatically generate an invoice for this contract.

The **Invoice** table stores individual monthly billing records. Each invoice is linked to a contract and records the total amount due, the breakdown of electricity cost, water cost, other costs, the due date, the actual payment date, and a status (**UNPAID**, **PAID**, **OVERDUE**, or **CANCELLED**). A unique invoice number is generated for each record to allow tenants and landlords to identify payments easily.

The **RentRequest**, **Incident**, and **Violation** tables support the operational side of the platform. **RentRequest** stores inquiries from public visitors who are interested in a room. **Incident** stores maintenance or repair requests submitted by tenants, each with a priority level (**LOW**, **MEDIUM**, or **HIGH**) and a resolution status. **Violation** stores rule infractions recorded by the landlord against specific tenants.

4.3.3 Multi-Tenant Data Privacy

Because multiple landlords share the same **PostgreSQL** database instance, preventing cross-tenant data leakage is a critical design concern. To solve this, every business table — **Room**, **Contract**, **Invoice**, **RentRequest**, **Incident**, and **Violation** — includes a `landlordId` column with a database-level index. When a landlord logs in, their `landlordId` is embedded in the **JWT** token. Every service method that queries these tables automatically appends a `WHERE landlordId = ?` filter using the value extracted from the authenticated request object. This means that even if a landlord attempts to request another landlord's invoice by guessing its UUID, the query returns no results because the `landlordId` filter does not match.

4.4 Authentication and Authorization

Security is a critical part of the platform. The system must ensure that only registered and approved users can access private pages, and that landlords can only access their own rental data. To achieve this, the backend implements token-based authentication and a layered role and ownership authorization system.

4.4.1 User Authentication with JWT

The authentication workflow uses **JSON Web Tokens (JWT)** to issue and verify user identities. The detailed execution steps are as follows:

1. The user submits their email and password to the `POST /auth/login` endpoint.
2. The **AuthService** queries the **User** table for a record with the matching email. If no record is found, an unauthorized exception is thrown immediately.
3. The service calls **bcrypt.compare** to verify the submitted password against the stored hash. If the comparison fails, an unauthorized exception is thrown.
4. If the credentials are valid, the service calls **JwtService.sign** to generate a signed **JWT** token. The token payload includes the user's `UUID (sub)`, email, role, and `landlordId`, as described in Table 4.1.
5. The signed token is returned to the client. The frontend stores it in memory and attaches it as a Bearer token to the Authorization header of every subsequent API request using an **Axios** interceptor.

Claim Key	Data Type	Purpose and Description
sub	String (UUID)	The unique identifier of the authenticated User.
email	String	The email address used to identify the user account.
role	Enum	The system role of the user (ADMIN, LANDLORD, or TENANT).
landlordId	String (UUID)	The landlord identifier used to scope all database queries for multi-tenancy.

Table 4.1: Structure of the JSON Web Token (JWT) Payload

Implementation Highlights:

- **Passport JWT Strategy:** Token validation is implemented as a **NestJS Passport** strategy. When a protected request arrives, the strategy extracts the token from the Authorization header, verifies its cryptographic signature using the server's secret key, and attaches the decoded payload to the request object as `req.user`. This decoded object is then available to all downstream guards and service methods without any additional database queries.
- **Password Hashing:** All passwords are hashed with **bcrypt** using a salt round of 10 before being saved to the database. Plain-text passwords are never stored or logged at any point.
- **Axios Interceptors on the Frontend:** The **Next.js** frontend uses a custom **Axios** instance with a request interceptor that reads the **JWT** token from the **Zustand** auth store and attaches it to every outgoing request automatically. This ensures that individual components never need to manage authentication headers directly.

4.4.2 API Protection with Auth Guards

To protect individual API endpoints, the backend implements two custom **NestJS** guards that are applied to controllers using decorators. As shown in Figure 4.3, every protected request passes through both guards before reaching the route handler.

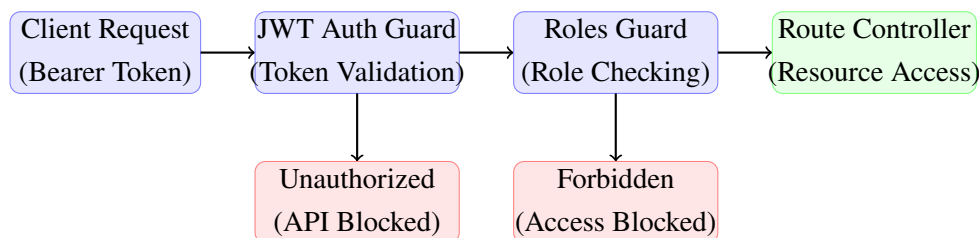


Figure 4.3: Authentication and Authorization Guard Flowchart

Implementation Highlights:

- **JwtAuthGuard:** This guard is the first checkpoint. It calls the **Passport JWT Strategy** to extract and verify the token. If the token is missing, malformed, or expired, the guard immediately throws an `UnauthorizedException` and the request never reaches the controller. On success, it attaches the decoded user payload to the request object.
- **RolesGuard:** This guard is applied after **JwtAuthGuard** on routes that require a specific role. It uses the **NestJS Reflector** to read the `@Roles()` decorator metadata on the current route handler. It then compares the `role` field from the decoded **JWT** payload against the required roles. If the user's role does not match (for example, a tenant trying to access a landlord-only route), the guard throws a `ForbiddenException`.
- **Public Route Bypass:** Routes that must be accessible without authentication (such as `GET /rooms/public`, `POST /auth/login`, and `POST /ai/public-chat`) are decorated with a custom `@Public()` metadata marker. The **JwtAuthGuard** checks for this marker using the **Reflector** and skips token validation for those routes entirely.

4.4.3 Multi-Tenant Authorization Filter

Apart from roles, the system enforces data ownership using the `landlordId` embedded in the **JWT** token. Because multiple landlords share the same database, the backend must prevent one landlord from reading or modifying another landlord's records.

Execution Flow:

1. When a landlord logs in, the **AuthService** fetches the landlord's `landlordId` from the **Landlord** table and embeds it in the **JWT** payload.

2. On every subsequent protected request, the **JwtAuthGuard** decodes the token and attaches the full payload — including `landlordId` — to `req.user`.
3. The service method receives the `landlordId` from `req.user` and passes it as a mandatory filter in every **Prisma** query: `where: { landlordId: req.user.landlordId }`.
4. The database returns only records that belong to that specific landlord, making it structurally impossible for one landlord to access another's data through the API.

Implementation Highlights:

- **No Separate Middleware:** The multi-tenant filter is not implemented as a separate middleware layer. Instead, the `landlordId` is injected directly into each service method call. This keeps the authorization logic co-located with the business logic and avoids any risk of the filter being accidentally bypassed by a new route.
- **Database Indexes:** All business tables have a database index on the `landlordId` column, defined in the **Prisma** schema using `@@index([landlordId])`. This ensures that the landlord-scoped filter queries remain fast even as the number of records grows.

To provide a clear interface for client applications, the Authentication Service exposes the following endpoints:

Table 4.2: Authentication Service API Specification

Endpoint	Method	Request Body	Role Required	Description
<code>/auth/register</code>	POST	<code>email, password, name, phone, role</code>	Public	Creates a new user account with PENDING status.
<code>/auth/login</code>	POST	<code>email, password</code>	Public	Validates credentials and returns a signed JWT token.
<code>/auth/profile</code>	GET	None	Authenticated	Returns the profile of the currently authenticated user.

4.4.4 Room and Tenant Management

Room management is the foundation of the platform. Before contracts or invoices can be created, the landlord must register rooms and approve tenant accounts. The room creation workflow validates all required fields on the backend using **NestJS** class-validator DTOs before saving to the database.

Execution Workflow — Room Image Upload:

1. The landlord selects an image file in the dashboard and submits it to `POST /rooms/:id/images`.

2. The **NestJS** controller receives the file via **Multer** middleware.
3. The **RoomService** calls the **CloudinaryService**, which uploads the binary file to the **Cloudinary** cloud using the official SDK.
4. **Cloudinary** returns a secure HTTPS URL for the uploaded image.
5. The service appends this URL to the room's `imageUrls` array in **PostgreSQL** using a **Prisma** update.
6. The updated room record is returned to the frontend, which immediately reflects the new image in the room card.

Implementation Highlights:

- **Public Room Listing:** The `GET /rooms/public` endpoint is decorated with `@Public()` and requires no authentication. It queries only rooms with `status: VACANT` across all landlords to populate the public landing page. This is the only cross-tenant query in the system and is intentionally scoped to vacant rooms only to prevent privacy leakage.
- **Tenant Account Status:** New users who register with the **TENANT** role receive **PENDING** status. The landlord must explicitly call `PATCH /tenants/:id/status` to set the status to **APPROVED** before the tenant can log in and access the portal. This prevents unauthorized users from accessing private rental data.

Table 4.3: Room Service API Specification

Endpoint	Method	Request Body / Params	Role Required	Description
<code>/rooms/public</code>	GET	None	Public	Returns all vacant rooms for the public listing page.
<code>/rooms</code>	GET	None	Authenticated	Returns all rooms belonging to the authenticated landlord.
<code>/rooms</code>	POST	roomNumber, floor, price, description	Landlord/Admin	Creates a new room record.
<code>/rooms/:id</code>	PATCH	Room fields to update	Landlord/Admin	Updates existing room details.
<code>/rooms/:id</code>	DELETE	None	Landlord/Admin	Deletes a room and all related records.
<code>/rooms/:id/images</code>	POST	image file	Landlord/Admin	Uploads a room image to Cloudinary and saves the URL.
<code>/rooms/:id/images</code>	DELETE	imageUrl	Landlord/Admin	Removes a room image from Cloudinary and the database.

Table 4.4: Tenant Service API Specification

Endpoint	Method	Request Body	Role Required	Description
/tenants	POST	userId, roomId	Landlord/Admin	Registers a user as a tenant under this landlord.
/tenants	GET	None	Landlord/Admin	Returns all tenants belonging to the landlord.
/tenants/:id	DELETE	None	Landlord/Admin	Removes a tenant record.
/tenants/:id/status	PATCH	status	Landlord/Admin	Approves or rejects a tenant's account registration.

4.5 Automation and Business Logic

This section describes the technical implementation of the key business processes in the backend. To ensure data consistency and provide a reliable user experience, the system automates several workflows using database transactions, scheduled background tasks, and real-time Web-Socket messaging.

4.5.1 Rental Contract Creation

Creating a rental contract requires updating multiple database records atomically. The most critical requirement is that a room's status must change from **VACANT** to **OCCUPIED** at the exact same moment the contract record is saved. If these two operations are not atomic, a system crash between them could leave the database in an inconsistent state where a room is occupied but has no active contract, or a contract exists for a room that is still marked vacant.

Execution Workflow:

1. The landlord submits contract details (tenant, room, dates, deposit, and rental price) to POST /contracts.
2. The **ContractService** first validates that the selected room's landlordId matches the requesting landlord's ID, preventing a landlord from creating a contract for another landlord's room.
3. The service queries the **Room** table to confirm the room's current status is **VACANT**. If the room is already occupied, it throws a conflict exception and stops execution.
4. The service opens a **Prisma** interactive transaction. Inside the transaction, it executes two operations: (a) creating the new **Contract** record, and (b) updating the **Room** status to **OCCUPIED**.
5. If both operations succeed, the transaction is committed and the new contract is returned. If any operation throws an error, **Prisma** automatically rolls back both changes, leaving the database in its original clean state.

Activity Diagram - Create Rental Contract

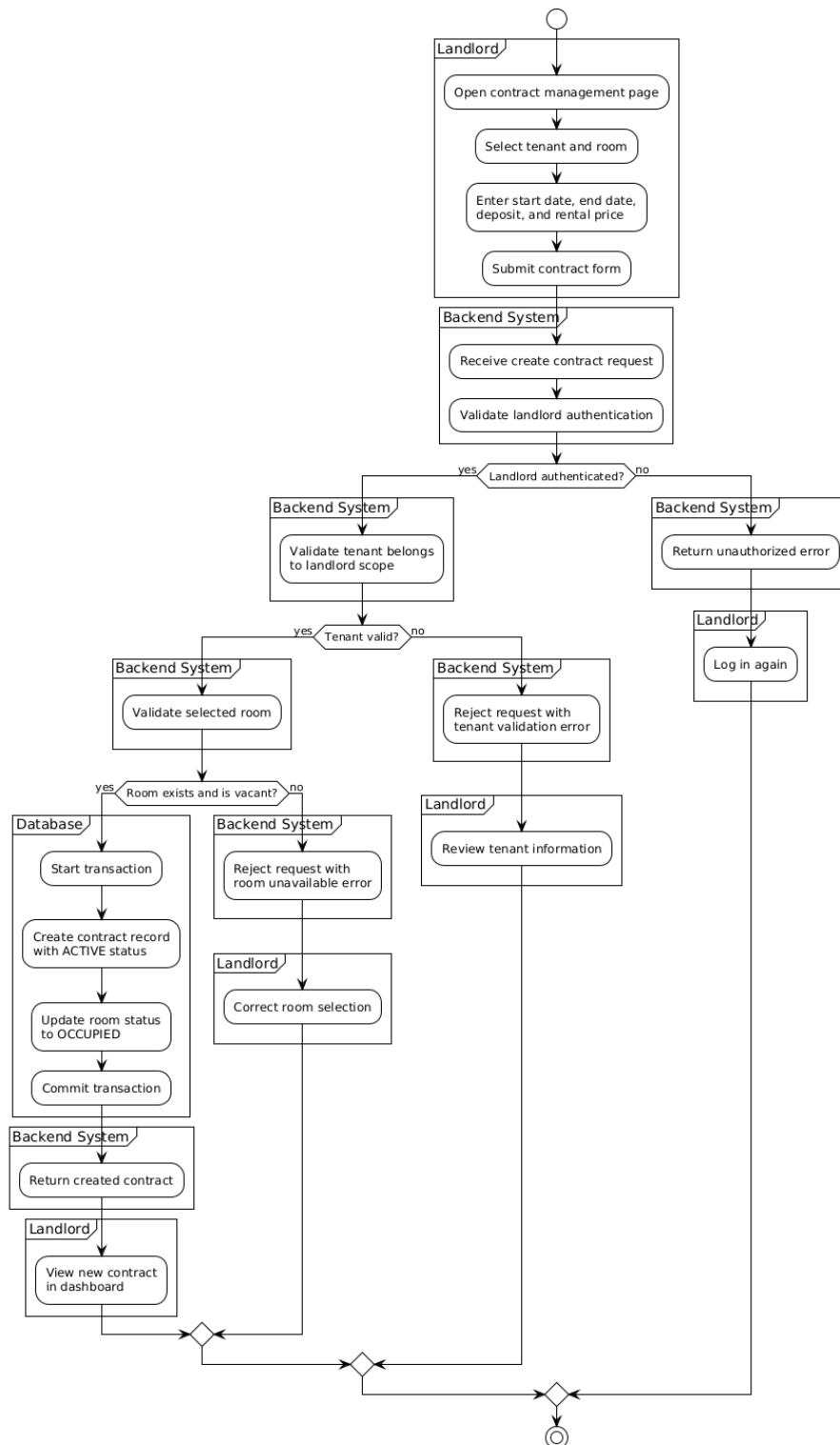


Figure 4.4: Activity Diagram - Contract Creation Process

Implementation Highlights:

- **Prisma Transactions:** The atomic update is implemented using **Prisma's** `$transaction()` API. This API accepts an array of Prisma operations and executes them inside a single

database transaction. This is the most reliable way to ensure that the contract and room status are always synchronized.

- **Contract Expiration Cron Job:** A separate **NestJS** scheduled task runs every day at midnight using the `@Cron()` decorator. It queries all contracts whose `endDate` has passed and whose status is still **ACTIVE**. For each expired contract, it updates the contract status to **EXPIRED** and updates the associated room status back to **VACANT** inside another Prisma transaction.
- **Startup Consistency Check:** When the backend application starts, a one-time bootstrap task runs to detect any rooms that are marked **OCCUPIED** but have no active contract (which could occur after an unexpected server crash). These rooms are automatically set back to **VACANT** to restore database consistency before the system begins serving requests.

Table 4.5: Contract Service API Specification

Endpoint	Method	Request Body	Role Required	Description
/contracts	POST	tenantId, roomId, startDate, endDate, deposit, rentalPrice	Authenticated	Creates a new contract and updates room status atomically.
/contracts	GET	None	Authenticated	Returns all contracts scoped to the authenticated user.
/contracts/:id/status	PATCH	status	Authenticated	Updates contract status (e.g., TERMINATED , EXPIRED).
/contracts/:id/autopay	PATCH	autoPayEnabled	Authenticated	Toggles automatic monthly invoice generation for the contract.
/contracts/:id	DELETE	None	Authenticated	Deletes a contract record.

4.5.2 Invoice Payment and Verification

The monthly billing cycle is managed through two mechanisms: an automated cron job that creates invoices, and a structured tenant-initiated payment flow.

Execution Workflow:

1. At the beginning of each month, a **NestJS** `@Cron()` task runs automatically. It queries all **Contract** records with status **ACTIVE** and `autoPayEnabled = true` and creates a new **Invoice** record for each one, pre-filling the rental price and setting the due date to the end of the current month.

2. The tenant logs in to the portal and navigates to the invoice list. The frontend calls `GET /invoices` to fetch all invoices linked to the tenant's active contract.
3. The tenant clicks *Pay* on an unpaid invoice. The frontend calls `GET /pdf/invoice/:id`, which triggers the backend to fetch the landlord's bank account details, call the **VietQR API** with the amount and account details, receive a QR code image in return, embed it into a **PDF** document, and return the file to the browser.
4. The tenant scans the **VietQR** code with their banking application and completes the bank transfer.
5. After transferring, the tenant clicks the *Already Transferred* button. The frontend calls `POST /invoices/:id/pay`.
6. The **InvoiceService** updates the invoice status to **PAID** and records the current timestamp in the `paidDate` field.
7. Simultaneously, the backend emits a **Socket.IO** event to the landlord's WebSocket room, pushing a real-time notification to the landlord's dashboard so they are immediately informed of the payment.

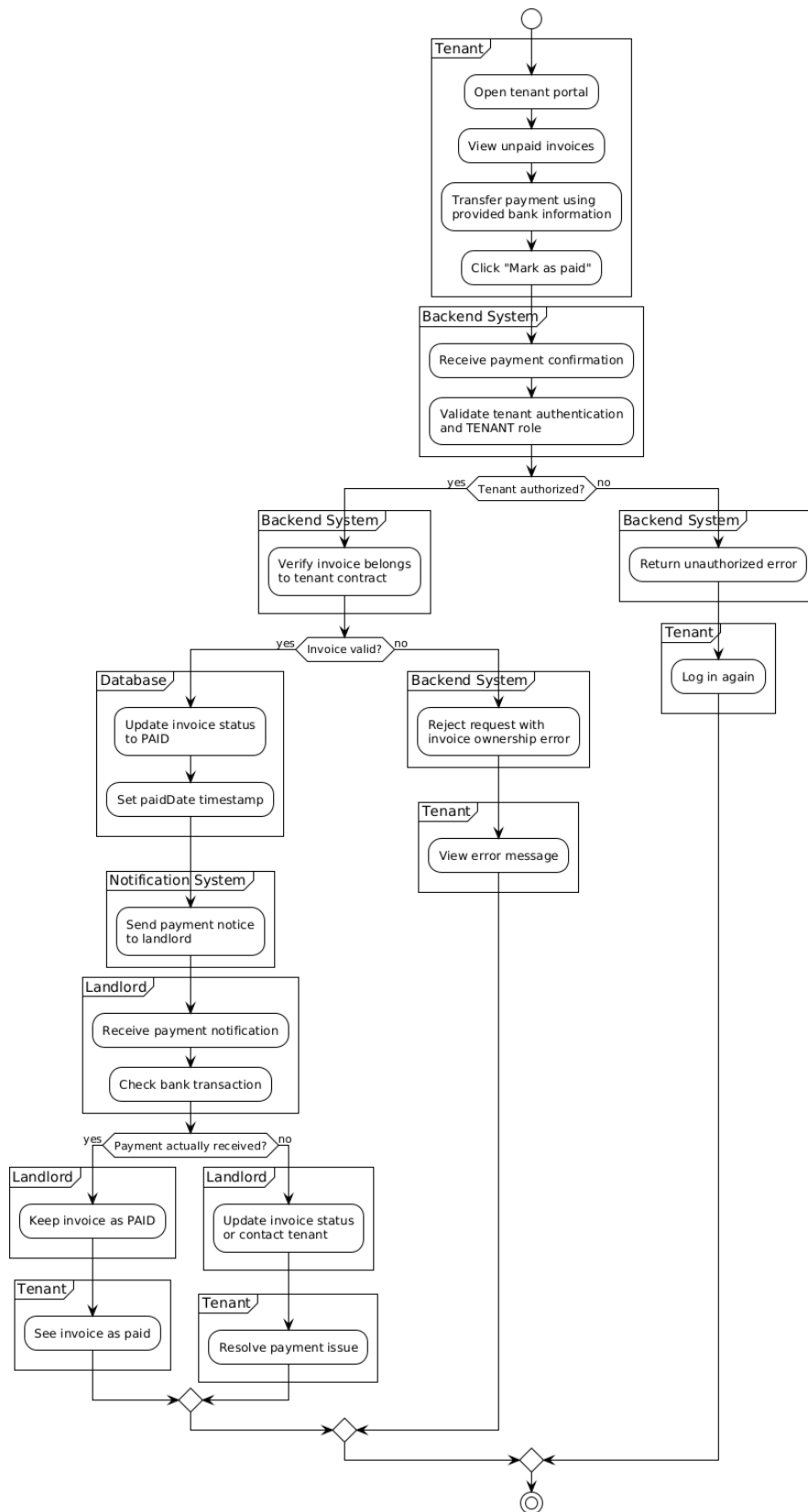
Activity Diagram - Invoice Payment and Verification

Figure 4.5: Activity Diagram - Invoice Payment and Verification

Implementation Highlights:

- **VietQR Code Generation:** The **VietQR** image API is called at PDF generation time with the landlord's bank name, account number, account holder name, and the exact invoice amount as query parameters. The API returns a PNG image of the QR code, which is then embedded into the PDF using the **PDFKit** library. The transfer memo is also pre-filled automatically with the tenant's name and the invoice month, so the landlord can identify payments in their bank statement without any manual cross-referencing.
- **Socket.IO Notification Gateway:** The **NotificationGateway** is a **NestJS** WebSocket gateway decorated with `@WebSocketGateway()`. When a client connects, the gateway authenticates the connection by verifying the **JWT** token sent in the WebSocket handshake. It then adds the client's socket to a named room identified by the landlord's ID (e.g., `landlord-abc123`). When a payment event occurs, the backend emits directly to this room, ensuring the notification reaches only the correct landlord.
- **Role-Gated Pay Endpoint:** The `POST /invoices/:id/pay` endpoint is restricted to the **TENANT** role using the **RolesGuard**. This prevents a landlord from accidentally marking their own invoices as paid from the admin panel. Landlords must use the `PATCH /invoices/:id/status` endpoint to update invoice status manually after verifying the bank transfer.

Table 4.6: Invoice Service API Specification

Endpoint	Method	Request Body	Role Required	Description
<code>/invoices</code>	POST	<code>contractId</code> , <code>amount</code> , <code>dueDate</code> , <code>utilities</code>	Landlord/Admin	Manually creates a new invoice for a contract.
<code>/invoices</code>	GET	None	Authenticated	Returns all invoices filtered by the authenticated user's role.
<code>/invoices/:id/status</code>	PATCH	<code>status</code>	Landlord/Admin	Manually updates invoice status after bank transfer verification.
<code>/invoices/:id/pay</code>	POST	None	Tenant	Marks invoice as paid and emits a Socket.IO notification to the landlord.

4.5.3 Tenant Incident Handling

Managing room repair requests requires tight coordination between tenants and landlords. The incident workflow is designed so that the landlord is notified instantly when a new issue is reported, and the tenant is notified in real time when their issue is resolved.

Execution Workflow:

1. The tenant opens the incident report form in the portal and submits a title, description, and priority level (**LOW**, **MEDIUM**, or **HIGH**) to `POST /incidents`.
2. The **IncidentService** saves the new incident record to the database with status **PENDING** and associates it with the tenant's `landlordId`.
3. Immediately after saving, the service calls the **NotificationGateway** to emit a `WebSocket` event to the landlord's room, notifying the dashboard of the new incident in real time.
4. The landlord reviews the incident in the dashboard and calls `PATCH /incidents/:id/status` to update the status to **IN_PROGRESS**, indicating that a technician has been assigned.
5. Once the issue is resolved, the landlord updates the status to **RESOLVED**. The backend emits another `WebSocket` event, this time to the tenant's portal session, informing them that their request has been closed.

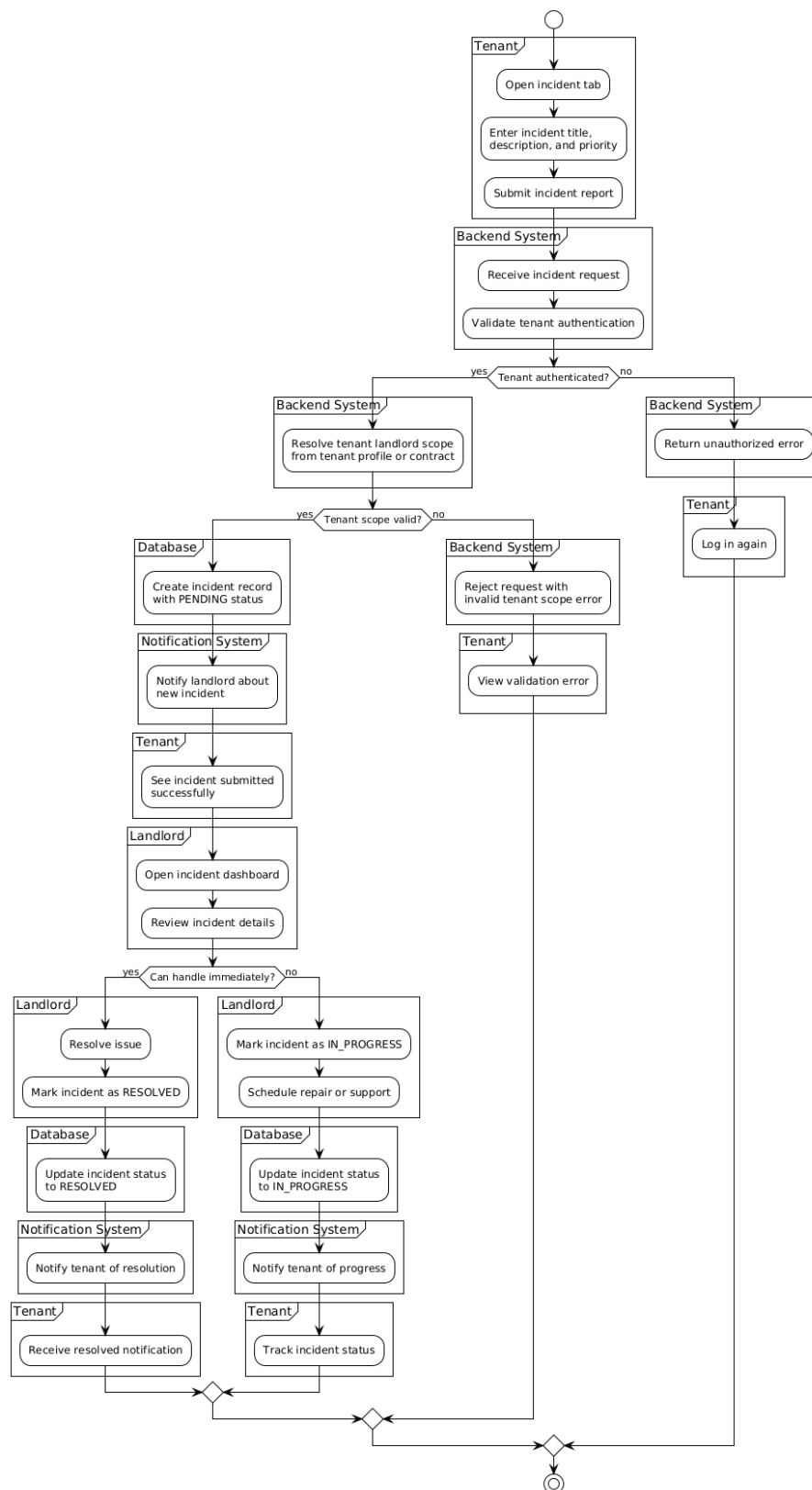
Activity Diagram - Tenant Incident Handling

Figure 4.6: Activity Diagram - Tenant Incident Handling

Implementation Highlights:

- **Priority Levels:** The priority field accepts three enum values: **LOW**, **MEDIUM**, and **HIGH**. This allows landlords to sort and filter incidents by urgency in the dashboard, so high-priority repairs (such as a water leak) are addressed before low-priority ones (such as a broken light switch).
- **Bidirectional Real-Time Notification:** Unlike invoice notifications, which only go from tenant to landlord, incident status updates flow in both directions. When the landlord updates the status, the backend checks whether the new status is **RESOLVED** and, if so, emits a notification event to the tenant's active WebSocket session as well.

The Incident Service and related operational services expose the following endpoints:

Table 4.7: Incident Service API Specification

Endpoint	Method	Request Body	Role Required	Description
/incidents	POST	title, description, priority, roomId	Authenticated	Submits a new incident report and notifies the landlord via WebSocket.
/incidents	GET	None	Authenticated	Returns all incidents scoped to the authenticated user's landlord.
/incidents/:id/status	PATCH	status	Authenticated	Updates incident resolution status and notifies the relevant party.

Table 4.8: Violations, Rent Requests, and Supporting Services API Specification

Endpoint	Method	Request Body / Params	Role Required	Description
/violations	POST	tenantId, description	Landlord/Admin	Records a new rule violation for a tenant.
/violations	GET	None	Tenant	Returns all violations visible to the current tenant.
/violations/tenant/:id	GET	None	Landlord/Admin	Returns all violations recorded for a specific tenant.
/violations/:id/resolve	PATCH	None	Landlord/Admin	Marks a violation as resolved.
/rent-requests	POST	name, email, phone, message, roomId	Public	Submits a room consultation request from a public visitor.
/rent-requests	GET	None	Landlord/Admin	Returns all rent consultation requests for the landlord.
/rent-requests/:id/status	PATCH	status	Landlord/Admin	Updates rent request status (CONTACTED or CANCELLED).
/pdf/invoice/:id	GET	None	Authenticated	Generates and returns the invoice PDF with the VietQR payment code.
/upload/image	POST	image file	Authenticated	Uploads an image to Cloudinary and returns the secure URL.

4.6 AI Integration

The platform integrates **Google Gemini** to automate administrative writing tasks and to provide intelligent self-service assistants for both public visitors and registered tenants. The key design principle is that the AI model is never asked to guess or invent information — it is always given real, live data from the database as structured context before answering any question.

4.6.1 AI Assistant with Google Gemini

The AI service is built using the official **Google Generative AI** Node.js library targeting the **Gemini 2.0 Flash Lite** model. This model is chosen because it has a very fast response time, making it suitable for real-time chat interactions. The service implements three distinct operating modes, each targeting a different user role and injecting different database context, as summarized in Table 4.9.

Assistant Mode	Database Context Injected	Core Capability
Notice Generator	None (Direct prompt input)	Generates polite, formal boarding house announcements in Vietnamese.
RentBot (Public)	Room number, rental price, floor, status, description.	Answers visitor questions about vacant rooms and directs them to register.
Tenant Assistant	Contract dates, monthly rent, deposits, unpaid invoices, violations, incidents.	Provides personalized rental Q&A, checks bills, and explains rules.

Table 4.9: Comparison of AI Assistant Modes and Data Contexts

Execution Workflow — Tenant Personal Assistant:

1. The authenticated tenant sends a message to `POST /ai/tenant-chat`.
2. The **AiService** extracts the tenant’s `userId` from the **JWT** token in the request.
3. It queries the database to fetch the tenant’s active contract, their full invoice history, all open incidents, and any recorded violations.
4. The service formats this data into a structured plain-text system prompt block describing the tenant’s rental situation.
5. It combines the system prompt with the tenant’s chat message and sends the assembled prompt to the **Gemini** model via the **generateContent** API call.
6. The model generates a response based on the injected context, and the service returns the response text to the frontend, which displays it in the chat interface.

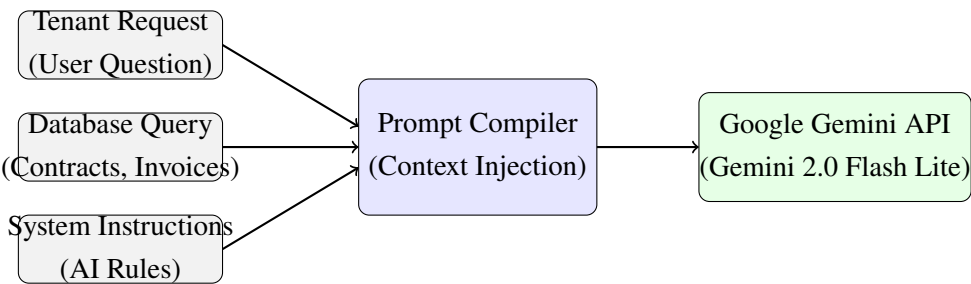


Figure 4.7: Contextual AI Prompt Construction Flowchart

Implementation Highlights:

- **Context Injection to Prevent Hallucination:** The most important design decision in the AI integration is that all three assistant modes inject real database records as plain-text context into the system prompt before the user's question is passed to the model. This prevents the model from inventing room prices, contract terms, or billing amounts. For the RentBot public assistant, the backend fetches the current list of vacant rooms and formats it as a structured list. For the tenant assistant, it fetches the specific tenant's contract and invoice data. The model is instructed in the system prompt to answer only based on the provided context and to decline questions outside of its scope.
- **Stateless Single-Turn Design:** The current implementation uses a stateless single-turn prompt model. Each API call sends the full context plus the current question, without maintaining a multi-turn conversation history. This keeps the implementation simple and avoids the cost of storing and retrieving conversation state between requests. It is sufficient for the current use cases, where each question is typically self-contained.
- **Notice Generator Mode:** For landlord notice generation, no database context is injected. The landlord types a short informal instruction (for example, "*remind tenants to pay by the 5th*"), and the system prompt instructs the model to rewrite it as a formal, polite Vietnamese boarding house announcement. This automates a task that previously required landlords to write formal notices manually.

The AI Service exposes the following endpoints for the three assistant modes:

Table 4.10: AI Service API Specification

Endpoint	Method	Request Body	Role Required	Description
/ai/generate-notice	POST	prompt	Authenticated	Generates a formal Vietnamese announcement from a short landlord note.
/ai/public-chat	POST	message	Public	RentBot answers visitor questions using live vacant room data as context.
/ai/tenant-chat	POST	message	Authenticated	Personal assistant answers tenant questions using their contract and invoice data.
/ai/chat	POST	message	Authenticated	General AI chat endpoint for landlord operational queries.

Chapter 5

RESULTS AND DISCUSSION

This chapter presents the results of implementing the *Smart Boarding House Management System* by examining the actual working prototype. Each section discusses a core feature through its real user interface, describing what was built, how it works from the user's perspective, and how it connects to the technical components described in Chapter 4.

5.1 Landlord Administration Dashboard and Room Management

The landlord administration dashboard is the central control point for property managers. After logging in, landlords are directed to the room management page, which provides an immediate summary of their entire property portfolio. As shown in Figure 5.1, the top of the page displays four statistics cards that give an instant overview of the property state: the total number of rooms, rooms that are currently available for rent, rooms that are currently occupied by active tenants, and rooms under maintenance. In the example shown, the landlord manages eight rooms in total, with four rooms available, four rooms occupied, and zero rooms under maintenance. A progress bar beneath the statistics shows the occupancy rate at fifty percent.

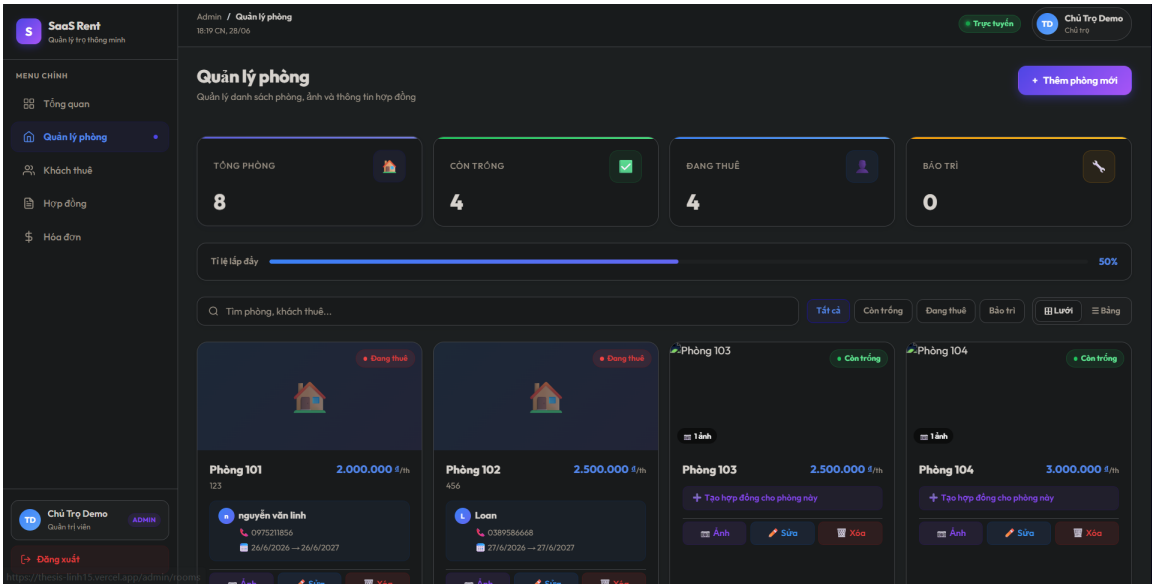


Figure 5.1: Landlord Administration Dashboard — Room Management Overview

Below the statistics, the room list is displayed as a grid of cards. Each card shows the room number, the monthly rental price, and the current room status label. The status label uses colour-coded tags so that landlords can identify available and occupied rooms at a glance without reading individual details. A filter bar above the grid allows landlords to switch between viewing all rooms, only vacant rooms, only occupied rooms, or only rooms under maintenance. This filter makes it faster to locate specific rooms, especially when managing a large number of properties.

Each room card also contains quick-action buttons so that landlords can navigate directly to that room’s detail, edit its information, or perform other operations without returning to the main menu. A prominent *Thêm phòng mới* (Add New Room) button is placed in the top-right corner of the page, allowing landlords to register a new room at any time. When a landlord creates or updates a room, the system validates all required fields on the backend before saving. Room images are uploaded to the **Cloudinary** cloud service and the secure image URLs are stored in the **PostgreSQL** database, which ensures that room photos load quickly for public visitors without consuming local server storage.

5.2 Public Room Listing and RentBot AI Chatbot

The public-facing listing page is the entry point for people searching for a room to rent. This page is accessible without any login and is designed to attract potential tenants by presenting available rooms clearly. As shown in Figure 5.2, the listing page displays a grid of room cards, each showing the room number, the monthly rental price in Vietnamese dong, and a *CÒN TRỐNG* (Available) tag that confirms the room is currently unoccupied. In the example, three rooms are visible: Phòng 103 at 2,500,000 VND, Phòng 104 at 3,000,000 VND, and Phòng 203

at 3,500,000 VND. Each card has a *Đăng ký phòng ngay* (Register for this room now) button that leads visitors to the room application form.

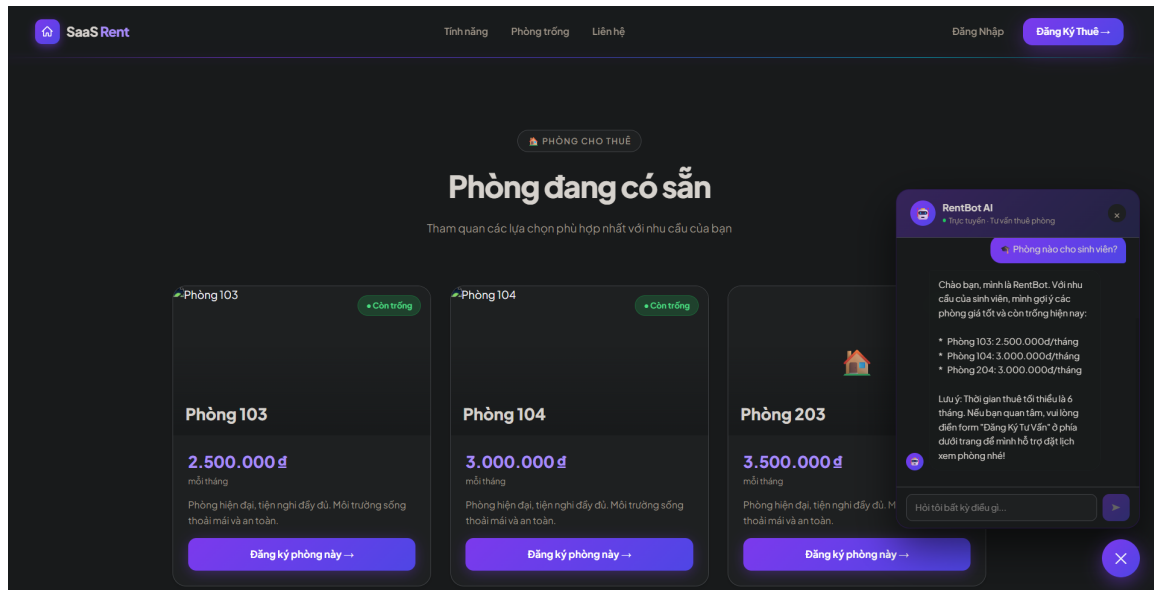


Figure 5.2: Public Room Listing Page with RentBot AI Chatbot

The most notable feature on this page is the **RentBot** chatbot widget, visible as a float-panel on the right side of the screen. RentBot is a context-aware **AI** assistant powered by the **Google Gemini** model. When a visitor opens the chatbot, the backend first queries the **PostgreSQL** database to fetch the current list of available rooms, including their room numbers, prices, floors, and descriptions. This real-time room data is then inserted into the system prompt as context before the visitor's question is sent to the **Gemini** model. As demonstrated in the screenshot, RentBot successfully responds to the visitor by listing the available rooms with their correct prices and advising the visitor to use the registration button to apply. Because the assistant reads from live database records, it can never provide incorrect or outdated room information. This approach directly addresses the common problem of prospective tenants receiving inconsistent room availability information.

5.3 Tenant Portal and Personal AI Assistant

Once a tenant has been registered by a landlord and logs in to the system, they are directed to the tenant portal. The tenant portal is a private space where each tenant can monitor all aspects of their rental relationship. As shown in Figure 5.3, the portal greets the tenant by name and displays the current date at the top of the page, providing a personalized and welcoming experience.

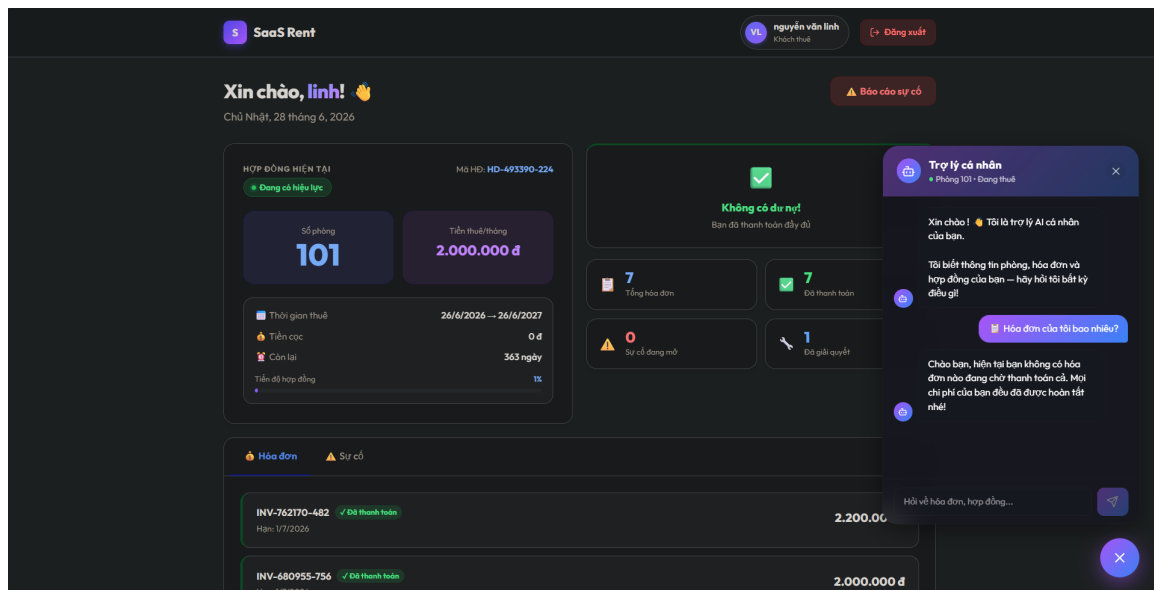


Figure 5.3: Tenant Portal Dashboard with Contract Summary and Personal AI Assistant

The left section of the portal displays the tenant's current active contract in a summary card labelled *Hợp đồng hiện tại* (Current Contract). This card shows the contract status, the room number the tenant is renting (Room 101 in this example), the rental start and end dates, and the number of days remaining on the contract. In the example shown, the tenant has a contract running from 24 June 2025 to 24 June 2027, with 363 days remaining. The tenant can also see the deposit amount and the monthly rental fee at a glance. Below the contract summary, the portal displays a list of recent invoices. Each invoice entry shows the invoice reference number, its payment status (such as *Chờ thanh toán* meaning Pending, or *Đã thanh toán* meaning Paid), and the invoice amount. This gives tenants a complete, up-to-date view of their billing history without needing to contact the landlord.

On the right side of the portal, a summary panel shows the tenant's activity counts, including the number of invoices already paid and the number of reported incidents that have been resolved. Two action buttons are placed prominently in the top-right corner: a *Báo cáo sự cố* (Report Incident) button for submitting maintenance requests, and the *Trợ lý cá nhân* (Personal Assistant) button that opens the AI chatbot.

The personal AI assistant is the most advanced feature of the tenant portal. When the tenant clicks the assistant button, the backend service queries the database to retrieve the tenant's active contract details, their full invoice history, any open incidents, and any recorded violations. All of this personal data is formatted as a structured system context and sent to the **Google Gemini** model together with the tenant's question. As demonstrated in the screenshot, the assistant can answer specific questions about the tenant's rental situation, such as informing the tenant about upcoming payment deadlines, explaining the amount still owed, or confirming whether an incident has been resolved. This level of personalization allows tenants to get precise, factual answers without calling or messaging the landlord directly.

5.4 Invoice Payment with VietQR Integration

The invoice payment workflow is one of the most practically important features of the system for both landlords and tenants. When a tenant views a pending invoice and clicks the payment button, the system opens a dedicated payment modal, as shown in Figure 5.4. This modal presents all the information a tenant needs to complete a bank transfer, without requiring them to search for the landlord's bank details manually.

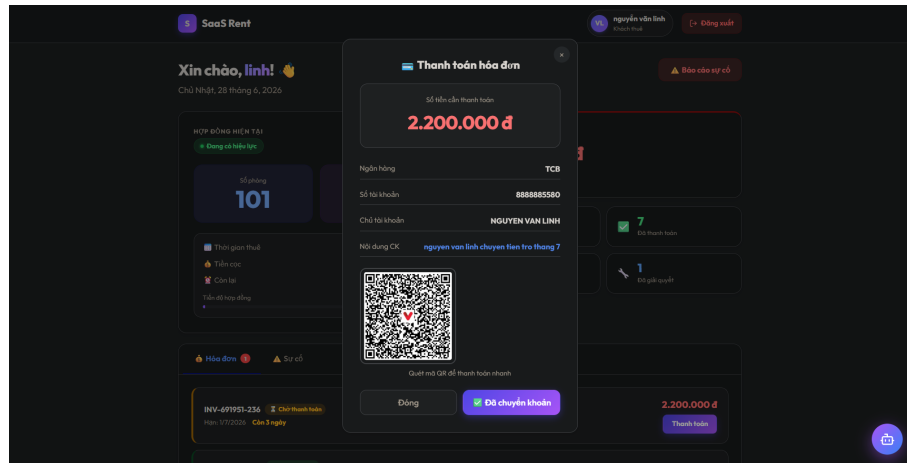


Figure 5.4: Invoice Payment Modal with VietQR Code and Bank Transfer Details

At the top of the modal, the exact amount to be paid is displayed in large text, making it immediately clear to the tenant. In the example, the tenant must pay 2,200,000 VND for the current month. Below the amount, the modal shows the complete bank transfer details: the bank name (Techcombank, abbreviated as TCB), the bank account number (8888885580), and the account holder name (NGUYEN VAN LINH). These details are retrieved from the landlord's profile stored in the database.

The most significant element of the payment modal is the automatically generated **VietQR** code displayed in the centre. This QR code is produced dynamically by the **VietQR** image API using the landlord's bank account information and the specific invoice amount. The transfer memo (Nội dung CK) is also pre-filled automatically to include the tenant's name and the invoice month, so the landlord can easily identify which payment belongs to which tenant and which month. In the example, the memo reads *nguyễn văn linh chuyển tiền trả tháng 7* (Nguyen Van Linh transfers rent for month 7). The tenant only needs to open their mobile banking application and scan this QR code to complete the full bank transfer in seconds.

After transferring the money, the tenant clicks the *Đã chuyển khoản* (Already Transferred) button at the bottom of the modal. This action sends a request to the backend, which immediately updates the invoice status to **Paid** and records the payment timestamp in the database. Simultaneously, the backend broadcasts a real-time **Socket.IO** WebSocket notification to the landlord's active dashboard session, informing them that a payment has been submitted. The

landlord can then open their bank application separately to verify that the transfer was received, and manually confirm or adjust the invoice status if necessary. Table 5.1 provides a full summary of all core features implemented in the prototype.

Feature	Status	Result
Room CRUD and image upload	Implemented	Landlords manage room records with photos stored on Cloudinary.
Public room listing	Implemented	Vacant rooms are shown on the public page for prospective tenants.
RentBot public chatbot	Implemented	AI assistant answers room availability questions using live database context.
Contract creation	Implemented	Contracts are created with atomic room status synchronization.
Monthly invoice generation	Implemented	Scheduled cron task creates unpaid invoices for all active contracts.
VietQR payment modal	Implemented	Payment modal displays bank details and a dynamically generated QR code.
Invoice payment tracking	Implemented	Tenants mark invoices paid; landlords receive real-time notifications.
Tenant portal dashboard	Implemented	Tenants view their contract summary, invoice history, and incident status.
Personal AI assistant	Implemented	AI assistant answers tenant questions using personal contract and invoice data.
Incident reporting	Implemented	Tenants submit repair requests; landlords manage and resolve them.
Real-time notifications	Implemented	Socket.IO broadcasts events to the landlord dashboard without page reload.

Table 5.1: Summary of Implemented System Features

5.5 Difficulties and Limitations

Although the prototype successfully implements all planned core features, several limitations must be acknowledged before the system can be considered ready for production deployment.

Lack of automated testing. The current codebase does not include comprehensive unit tests or integration tests. Automated testing is especially important for verifying complex business workflows such as the contract creation transaction, the monthly invoice generation cron job,

and the multi-tenant authorization filters. Without these tests, adding new features in the future carries a higher risk of accidentally breaking existing behaviour.

Manual payment verification. The invoice payment workflow requires the landlord to verify bank transfers manually by checking their bank application separately. Because the system is not connected to a live banking webhook or an open banking API, it cannot automatically detect and confirm incoming payments. The tenant must click the *Already Transferred* button to update the invoice status, and the landlord must independently confirm the transfer. This introduces a risk of human error and delays in payment confirmation.

Dependency on external AI service. All three **AI** assistant features — the notice generator, the public RentBot, and the personal tenant assistant — depend entirely on the external **Google Gemini** API. If the Google service is temporarily unavailable, rate-limited, or slow, the chatbot features will fail or respond with significant delays, directly reducing the reliability of the system for users who depend on it.

Multi-tenant security relies on application-level filtering. Data privacy between landlords is enforced by filtering every database query using the landlord identification code extracted from the **JWT** token. While this approach works correctly in the current implementation, a production-grade SaaS platform should also implement database-level row security policies and audit logging to guard against potential code-level mistakes that could expose one landlord's data to another.

No mobile application. The platform is built as a responsive web application accessible through a browser. However, tenants and landlords in Vietnam predominantly use smartphones for day-to-day activities, and a dedicated mobile application would provide a significantly smoother experience, especially for scanning QR codes and receiving push notifications.

5.6 Discussion

The results of this project demonstrate that a fully integrated web-based boarding house management platform can be built using modern open-source technologies and cloud services within the scope of a single undergraduate thesis project. By combining a modular **NestJS** backend, a server-rendered **Next.js** frontend, a **PostgreSQL** relational database, and the **Google Gemini** AI model, the system delivers a coherent set of features that directly address the real-world problems described in Chapter 1.

The most significant technical achievement is the integration of multiple system components into seamless end-to-end workflows. For example, the room status in the database changes automatically when a contract is created, expires, or is terminated. Monthly invoices are generated without any manual intervention from the landlord through the scheduled cron job. When a tenant submits payment confirmation, the landlord's dashboard is updated instantly through the **Socket.IO** WebSocket connection without requiring a page refresh. These automated workflows eliminate manual data entry, reduce the chance of human error, and save considerable time for

landlords managing multiple rooms simultaneously.

The **AI** integration proves to be one of the most practical additions to the system. The public **RentBot** reduces the number of repetitive inquiries that landlords must answer by allowing visitors to ask questions about available rooms directly on the listing page. The personal tenant assistant gives tenants a self-service channel for checking their contract terms, outstanding bills, and incident status at any time, without needing to contact the landlord. Both assistants use live data from the database rather than static scripts, which ensures that the answers are always accurate and up-to-date.

The multi-tenant architecture design ensures that the platform can serve multiple landlords from a single deployed instance without any risk of data leakage between users. Each landlord's rooms, tenants, contracts, and invoices are fully isolated through consistent use of the landlord identification code as a query filter on every database operation. This design is scalable and aligns with the SaaS business model described in the objectives of this thesis.

In conclusion, the implemented prototype successfully validates the design decisions made in Chapter 4 and fulfills the functional requirements defined in Chapter 3. The limitations identified — particularly the absence of automated tests, manual payment verification, and external AI dependency — are well-understood trade-offs that are appropriate for a prototype at this stage. Addressing these limitations through automated testing, open banking API integration, and AI fallback strategies would be the natural next steps for transforming this prototype into a production-ready product.

Chapter 6

CONCLUSION AND FUTURE WORK

This chapter summarizes the final results of this bachelor thesis and outlines the main future directions to improve the system.

6.1 Conclusion

The *Smart Boarding House Management System* successfully provides a practical foundation for digitizing rental property operations. The project solves common problems in boarding house management by centralizing room details, tenant registration, contracts, invoices, payment tracking, incidents, violations, rent requests, notifications, and AI support in a single **SaaS** platform.

From a software engineering perspective, the project demonstrates that combining modular design with clean architecture rules makes codebases highly maintainable. The backend built with **NestJS** and **Prisma ORM** ensures that database operations in **PostgreSQL** are secure and consistent. The use of custom guards like the **JwtAuthGuard** and **RolesGuard** protects private API endpoints. On the frontend, **Next.js** and **Zustand** provide a modular portal interface. The integration of supporting services like **Socket.IO** for notifications, **Cloudinary** for uploads, and **Google Gemini** for AI chat completes a modern development package. This prototype represents a secure, maintainable, and solid base for a commercial rental application.

6.2 Future Work

Although the prototype works successfully, several technical improvements can be added in future versions to make it ready for production.

6.2.1 Production Payment Integration

The current payment verification process is manual. To automate this, the system should integrate payment gateway APIs such as **PayOS**, **MoMo**, or **VNPay** and listen to secure database

webhooks. When a tenant scans the **VietQR** code and transfers money, the payment provider will automatically send a secure HTTP post request to the backend. The backend will parse the webhook, verify the invoice ID, update the invoice status to paid, and send an instant notification to both the landlord and tenant, eliminating manual verification.

6.2.2 Advanced Utility Billing

The utility billing module can be improved by tracking consumption history. Future versions should save previous and current meter readings in the database, allowing the system to calculate the difference between readings and automatically compute the electricity or water cost. The system can also run consistency checks to warn the landlord if a room has abnormally high consumption compared to previous months.

6.2.3 Stronger Multi-Tenant Security

To prepare for large-scale SaaS deployment, database-level security must be hardened. Future work should implement **Row-Level Security (RLS)** in **PostgreSQL** to enforce isolation at the database layer. Additionally, the backend should include security auditing logs to record database queries, rate limiters using the NestJS throttler module to prevent automated attacks, and strict origin checks on WebSockets to secure real-time connections.

6.2.4 Mobile Tenant Experience

Tenants primarily use mobile phones to interact with the boarding house system. Developing a native mobile application using **React Native** or **Flutter**, or converting the frontend into a **Progressive Web App (PWA)**, will make it much easier for tenants to view bills and report incidents. This will also allow the system to send mobile push notifications directly to the tenant's lock screen when a new invoice is created.

6.2.5 Analytics and Forecasting

The landlord dashboard can be expanded with analytics to track business performance. The system can calculate metrics like monthly recurring revenue, overall occupancy rate, average repair resolution times, and outstanding debts. Furthermore, machine learning models or Gemini forecasting can analyze historical data to help landlords predict vacancy risks and manage cash flow.

6.2.6 Document and Contract Management

The document management features can be expanded to include paperless contract signing. The backend can generate PDF lease documents dynamically and integrate electronic signature ser-

vices like **DocuSign** or local digital signature certificates. This will allow landlords and tenants to sign contracts digitally, making them legally binding and securely stored in the system.

REFERENCES

- [1] Cloudinary. Cloudinary documentation, 2026. Accessed: 2026-06-27. URL: <https://cloudinary.com/documentation>.
- [2] Google DeepMind. Gemini models documentation, 2026. Accessed: 2026-06-27. URL: <https://ai.google.dev/gemini-api/docs>.
- [3] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, RFC Editor, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
- [4] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [5] Neon, Inc. Neon: Serverless postgres, 2026. Accessed: 2026-06-27. URL: <https://neon.tech/docs>.
- [6] NestJS. Nestjs documentation, 2026. Accessed: 2026-06-27. URL: <https://docs.nestjs.com/>.
- [7] Prisma. Prisma documentation, 2026. Accessed: 2026-06-27. URL: <https://www.prisma.io/docs>.
- [8] Render. Render cloud hosting documentation, 2026. Accessed: 2026-06-27. URL: <https://docs.render.com/>.
- [9] Socket.IO. Socket.io documentation, 2026. Accessed: 2026-06-27. URL: <https://socket.io/docs/>.
- [10] Vercel. Next.js documentation, 2026. Accessed: 2026-06-27. URL: <https://nextjs.org/docs>.
- [11] Vercel. Vercel documentation, 2026. Accessed: 2026-06-27. URL: <https://vercel.com/docs>.